

1. Abstract

Retail financial risk profiling is still performed almost exclusively by questionnaires scored against fixed, hand-written rubrics. Such rubrics are transparent and cheap, but their weights encode assumption rather than evidence, and being additive they cannot represent interactions between a respondent's attributes. This project replaces the rubric with a learned classifier while attempting to preserve the per-answer transparency that made it defensible, and reports what happened when that attempt was audited end to end.

A web application was built spanning data generation, preprocessing, model selection, explanation, persistence and presentation. Twenty-six demographic, financial and lifestyle attributes are elicited through a four-step form, validated by schemas shared between a React client and a FastAPI service, classified into three risk bands, attributed with Shapley values, and stored in PostgreSQL. Five classifiers were compared on an identical partition; a Random Forest was selected on weighted F1 and tuned, reaching accuracy 0.6039, weighted F1 0.6006 and one-vs-rest ROC AUC 0.7461 on 4,400 held-out records, with five-fold cross-validated F1 of 0.5996 ± 0.0032 .

Because no public dataset joins these attributes to a behavioural risk label, and because risk-band suitability is never observed in practice, the 22,000 records are synthetic and generated from a documented scoring function. This forfeits any empirical claim about real financial behaviour. It permits, in exchange, an evaluation unavailable for real data: since the label is a threshold on a linear score perturbed by Gaussian noise of standard deviation 8.0 against a signal of standard deviation 7.64, the Bayes-optimal accuracy can be derived exactly. It is 0.6104. The model therefore attains 98.9% of the maximum accuracy any classifier could reach, and 95.9% of the accuracy available above the 0.4500 majority-class baseline. Its predicted class distribution tracks the Bayes rule's rather than the true distribution, and 1,706 of its 1,743 errors fall between adjacent bands, with only 0.84% confusing Low with High. An accuracy of 60% read against unity suggests a weak model; read against its ceiling it indicates that nearly all recoverable signal was extracted.

A secondary classifier predicting investment preference is reported as a negative result. It scores 0.2586 against a majority baseline of 0.2869 and ranks at chance. Inspection of the generator shows the label is drawn from a distribution conditioned on the risk band and on no feature whatever, bounding any classifier at 0.3675 even under perfect knowledge of that band. The failure was diagnosed by reading the data-generation code rather than by further experimentation.

The most consequential findings concern defects that no accuracy metric exposed. Feature engineering had been implemented twice, once for training and once for serving, and the two disagreed in five places; because the test set was evaluated through the training path, every reported metric remained correct while every live prediction was wrong. A third of one categorical column was silently destroyed by the CSV reader and imputed to a different category. The explanation pipeline discarded the sign of every Shapley value before display, so the interface distinguished risk-increasing from risk-decreasing features over data in which no value was ever negative. The submission form posted to the non-persisting endpoint, so nothing a user entered reached the database. Five of these six defects lay not inside any function but in the agreement between two representations of one computation, and all eleven tests in the suite passed

throughout. They were found by comparing representations against each other: routing training records through the serving preprocessor and checking the feature vectors agree, which they now do to 10^{-16} , with prediction agreement on every held-out row.

Seven of ten objectives were met, two only after repair. Two were not: the explainability module does not deliver signed per-user attributions, and the investment-preference objective was unattainable by construction. The report treats these as results rather than omissions.

The contribution is a systems result, not a scientific one. It advances neither behavioural finance nor machine learning. It offers a verified end-to-end pipeline, an evaluation conducted against a derived ceiling rather than against perfection, and a documented account of the failure modes that conventional model evaluation cannot see — chief among them that the accuracy stated in a report and the accuracy a user receives are distinct quantities, and that establishing their equality requires a comparison nobody thinks to run.

Keywords — behavioural finance, financial risk profiling, explainable artificial intelligence, SHAP, Shapley values, Random Forest, Bayes-optimal accuracy, training–serving skew, synthetic data, negative results.

2. Introduction

2.1 Background

Classical finance assumes that people, faced with a choice among uncertain outcomes, maximise expected utility. The assumption is analytically convenient and empirically false in ways that are systematic rather than random. Kahneman and Tversky demonstrated that individuals evaluate outcomes as gains and losses relative to a reference point rather than as final states of wealth, that they weight losses roughly twice as heavily as equivalent gains, and that they distort small probabilities [1]. Preferences reverse when the same choice is described differently.

Behavioural finance grew out of that observation. Barberis and Thaler survey its two structural claims: that arbitrage is limited, so mispricing can persist, and that the psychology of real investors departs predictably from the rational agent of the textbook [2]. For an individual deciding how much to save, whether to hold equity, and how much debt to carry, the departures matter more than the equilibrium.

Financial institutions have long needed to place a client somewhere on a risk spectrum before recommending anything. The instrument they use is almost always a questionnaire scored by a fixed rubric. Grable and Lytton's thirteen-item scale is the best known and among the few to have been validated psychometrically [3]. Its persistence is a mark of how little the practice has changed.

2.2 Motivation

Two weaknesses of the rubric approach motivated this project.

The first is that the weights are asserted. An analyst decides that a question about reaction to a market decline is worth five points and that dependants subtract three, and nothing in the data confirms or refutes those numbers. A rubric cannot express the idea that a low savings rate is more serious for a person supporting a family than for one who is not, because a sum of independent terms has no way to represent an interaction.

The second is subtler and is documented across the industry literature. Questionnaires routinely conflate two different quantities: a person's *capacity* to absorb loss, which is an objective fact about their balance sheet, and their *tolerance* for it, which is a psychological disposition. Klement's survey of the field treats the confusion as endemic and notes that self-reported willingness to take risk is a poor guide to behaviour under actual loss [4].

Machine learning offers a partial answer to the first weakness. Where labelled data exists, the weights need not be asserted; they can be estimated. This is unremarkable in adjacent domains — credit scoring has used statistical classification for decades, and Lessmann and colleagues benchmark a large family of algorithms on that task [5]. It is less common in risk profiling, and the reason is not technical. A credit decision needs to be accurate. A risk profile needs to be *explicable* to the person it describes, and a learned model, unlike a rubric, does not explain itself.

The last decade closed that gap. Lundberg and Lee unified several attribution methods under the Shapley value from cooperative game theory [6], and a follow-up gave an exact polynomial-time algorithm for tree ensembles [7], which makes per-prediction attribution cheap enough to compute inside a web request.

What a rubric provides by construction — a per-answer contribution to the score — a learned model can now provide by computation.

2.3 Scope of This Work

This project builds an end-to-end system on that premise: elicit twenty-six demographic, financial and lifestyle attributes through a web form; classify the respondent into one of three risk bands with a trained model; attribute the outcome to the individual's own inputs with SHAP; persist the assessment; and present the result through a dashboard.

Two boundaries must be stated at the outset, because they determine how everything that follows should be read.

The data is synthetic. No survey was administered and no human respondent contributed a record. The 22,000 rows were produced by a generator with documented conditional dependencies, because no public dataset combines these attributes with a behavioural risk label. The accuracy reported in Section 2.3 measures how faithfully a learning algorithm recovers a known generating process. It is not evidence about the financial behaviour of real people, and no claim to the contrary is made anywhere in this report.

The system is not financial advice. It classifies. The recommendations it displays are produced by fixed rules over the user's inputs, not by the model, and they are illustrative.

2.4 Contributions

What this work offers is chiefly a systems result, together with three findings that emerged from examining the system rather than from designing it.

A complete, containerised pipeline runs from data generation through preprocessing, model selection, explanation, persistence and presentation, with the interfaces between its stages specified and verified.

Because the label's generating function is known, the **Bayes-optimal accuracy** for this classification can be computed rather than estimated. It is 0.6104. The tuned Random Forest attains 0.6039, or 98.9% of the ceiling — a figure that transforms an apparently mediocre 60% into evidence that the model has extracted nearly all recoverable signal. Reporting an accuracy against a derived ceiling, rather than against 1.0, is the methodological point this report presses hardest.

A **negative result** is reported in full. The secondary model predicting investment preference scores below its own majority-class baseline. Inspection of the generator shows the label was drawn from a distribution conditioned on nothing but the risk band, so no model could have succeeded. The diagnosis came from reading forty lines of data-generation code, not from further experiments.

A class of defect is characterised that **no accuracy metric can expose**. Feature engineering was implemented twice — once for training, once for serving — and the two disagreed in five places while every reported metric remained correct, because the test set was evaluated through the training path. Sculley and colleagues describe this family of maintenance hazards as the hidden technical debt of machine learning systems [8]. The repair here was structural: the training pipeline now emits every constant it fits, and the serving path reads them, so the two cannot diverge.

2.5 Organisation

Section 3 reviews the literature. Sections 4 to 7 establish the existing approach, the gap, the problem and the objectives. Section 8 presents the proposed system and Sections 9 to 11 its methodology and architecture. Sections 12 to 14 describe the dataset, the input parameters and the derived features. Sections 15 to 21 cover model selection, implementation and explainability. Sections 22 to 24 report and interpret the results. Sections 25 to 28 assess advantages, limitations and future work, and Section 29 lists references.

3. Literature Review

3.1 Departures from Expected Utility

The starting point of behavioural finance is a negative result about the standard model. Kahneman and Tversky's prospect theory replaces expected utility with a value function defined over gains and losses relative to a reference point, concave in gains, convex and steeper in losses, together with a weighting function that overweights small probabilities [1]. Two consequences bear directly on risk profiling. Loss aversion means a person's stated willingness to accept a gamble depends on how the gamble is framed, so a questionnaire's wording is not neutral. And reference dependence means risk attitude is not a fixed personal constant; it moves with recent outcomes.

Barberis and Thaler's survey organises the field around limits to arbitrage and investor psychology, and catalogues the biases that recur in the empirical literature: overconfidence, which inflates trading frequency; herding; mental accounting; and present bias in intertemporal choice [2]. Their treatment is relevant here mainly as a caution. These biases are established as population-level regularities, established through controlled experiment and market data. They are not established as attributes reliably inferable from a person's demographic and spending profile, which is what an automated system would need.

3.2 Measuring Risk Attitude

Grable and Lytton constructed and validated a thirteen-item instrument for financial risk tolerance, reporting its factor structure and internal consistency [3]. The scale remains a reference point three decades on, and its methodology — item development, pilot testing, psychometric validation — is what distinguishes an instrument from a form.

Klement's compilation for the CFA Institute Research Foundation surveys the state of practice and is unusually candid about its failures [4]. The recurring criticism is conceptual rather than statistical: instruments intended to measure *risk tolerance*, a psychological disposition, frequently measure *risk capacity*, an objective property of a balance sheet, or silently blend the two. A person with substantial savings and no dependants has high capacity regardless of how they feel about volatility. Conflating the two produces an assessment that is neither.

This distinction shapes the present work and its limits. The twenty-six parameters collected here are overwhelmingly capacity-side: income, expenses, debt, buffer, credit score. The lifestyle attributes were included on the reasoning that discretionary spending might reveal dispositions that self-report does not, since Kahneman and Tversky's framing effects imply that asking someone directly how much risk they tolerate is unreliable. Whether that reasoning holds cannot be tested on the data used here, for reasons Section 24.5 sets out.

3.3 Statistical Classification of Financial Risk

Applying supervised learning to financial risk is not novel. Credit scoring has done so since long before the term machine learning was common, and Lessmann and colleagues provide the definitive modern benchmark, comparing a large family of classifiers across multiple credit datasets and several performance

measures [5]. Their headline finding is that ensembles of decision trees are consistently among the strongest performers and that differences between the leading methods are often smaller than differences between evaluation protocols.

The algorithmic literature underlying those ensembles is well settled. Breiman's Random Forest grows decorrelated trees on bootstrap samples with feature subsampling at each split, and derives a bound on generalisation error in terms of individual tree strength and inter-tree correlation [9]. Gradient boosting fits trees sequentially against the ensemble's residual; XGBoost contributes a regularised objective and a sparsity-aware split-finding algorithm [10], LightGBM contributes gradient-based one-side sampling and histogram binning for speed [11], and CatBoost contributes ordered boosting and a principled treatment of categorical variables that avoids target leakage [12].

Whether such ensembles remain preferable to neural networks on tabular data has been examined directly. Grinsztajn, Oyallon and Varoquaux benchmark both families across a curated collection of tabular datasets and find tree-based models still ahead, attributing the gap to their robustness to uninformative features and to their bias toward axis-aligned, non-smooth decision boundaries [13]. This finding informed the model selection in Section 15, where a neural network was included as a control and lost by a wide margin.

3.4 Explanation

A learned classifier gives up the transparency of a rubric, and the explainability literature exists to buy some of it back.

Ribeiro, Singh and Guestrin's LIME explains an individual prediction by fitting an interpretable surrogate — typically sparse linear — to the model's behaviour in a neighbourhood of the instance [14]. The approach is model-agnostic, which is its strength, and locally approximate, which is its weakness: the explanation is of the surrogate, not of the model.

Lundberg and Lee showed that LIME and several other attribution methods are special cases of a single class of additive feature-attribution measures, and that within that class the Shapley value is the unique solution satisfying local accuracy, missingness and consistency [6]. This moves attribution from heuristic to axiomatic. The obstacle is cost: exact Shapley values require evaluating all feature subsets, which is exponential.

That obstacle disappears for tree ensembles. Lundberg and colleagues give a polynomial-time algorithm computing exact Shapley values for trees, and extend it to interaction effects and to global summaries built by aggregating local explanations [7]. This result is what makes the present system possible: it permits exact per-user attribution over a 200-tree ensemble inside a single HTTP request, without sampling.

A dissenting position deserves recording. Rudin argues that for high-stakes decisions one should not explain a black box but build an interpretable model instead, since a post-hoc explanation is by definition not the computation it purports to describe, and may be faithful in aggregate while misleading in particular [15]. The argument has force in the present setting. Section 21.4 documents an implementation in which the attribution reaching the user had its sign discarded — a post-hoc explanation that was not merely approximate but directionally uninformative, while continuing to look authoritative. Rudin's warning is that explanations invite exactly this kind of trust, and the experience here supports it.

3.5 Machine Learning Systems as Engineering Artefacts

The literature above concerns models. A deployed risk-assessment system is not a model, and the distinction turned out to be where this project's most instructive failures lived.

Sculley and colleagues frame the maintenance burden of production machine learning as technical debt, and enumerate its forms: entanglement, in which changing anything changes everything; undeclared consumers; data dependencies that no compiler checks; configuration debt; and the accumulation of glue code around a small modelling core [8]. Their central observation is that only a fraction of a real ML system is the learning algorithm, and that the surrounding infrastructure is where defects accumulate unobserved.

Section 24.3 of this report is, in effect, a case study in that claim. Five defects arose from a single feature computation being implemented twice, in two languages of expression, with no mechanism asserting their equivalence. Every unit test passed. Every reported metric was correct. Every live prediction was wrong. The literature on model evaluation offers no protection against this, because the fault is not in the model.

3.6 Position of This Work

The behavioural-finance literature establishes that risk attitude is unstable, frame-dependent and poorly measured by direct question. The measurement literature establishes that existing instruments conflate capacity with tolerance. The machine-learning literature establishes that tree ensembles are the appropriate estimator for tabular financial data, and that exact per-instance attribution for such models is computationally free. The systems literature establishes that the resulting artefact will fail in places the model literature never looks.

This project sits at the intersection, and its principal claims are correspondingly modest. It does not advance behavioural finance, because its data is synthetic. It does not advance machine learning, because it applies established methods. What it contributes is an end-to-end system in which the interfaces between these stages are specified and verified, an evaluation conducted against a derived Bayes ceiling rather than against perfection, and a documented account of the failures that the standard evaluation apparatus does not detect.

4. Existing System

4.1 The Rubric-Scored Questionnaire

The instrument in near-universal use for retail risk profiling is a short questionnaire whose answers are converted to a score by a fixed table of weights. A respondent answers between ten and twenty items; each response carries a preset point value; the values are summed; the total is compared against cut-offs that map it onto a small number of bands, typically labelled conservative, moderate and aggressive. An allocation is then recommended per band.

Grable and Lytton's thirteen-item scale is the canonical academic example and one of the few subjected to psychometric validation [3]. Commercial implementations, whether administered by an adviser or embedded in a robo-advisory onboarding flow, follow the same structure.

4.2 Why It Persists

The approach has genuine merits, and any replacement must account for them rather than dismiss them.

It is transparent. An adviser can point at a row of the scoring table and tell a client precisely how many points their answer contributed. This is not a post-hoc reconstruction of the model's reasoning; it *is* the model's reasoning, and the distinction matters when a client disputes an outcome or a regulator requests an audit trail.

It is cheap. No data collection, no training, no infrastructure. The rubric is a spreadsheet.

It is stable. The same answers always give the same score, and the score does not drift as a training population changes.

And it is defensible. Rudin's argument that high-stakes decisions should use inherently interpretable models rather than explained black boxes describes the rubric exactly [15]. The incumbent is not a naive baseline; it is the position the explainability literature would recommend on principle.

4.3 Limitations

Three weaknesses are structural rather than incidental.

The weights encode assumption, not evidence. Nothing in any dataset determines that a particular answer should be worth five points. The rubric expresses an analyst's beliefs about what constitutes risk, and where those beliefs are wrong the instrument is wrong in the same direction for every respondent it ever scores. There is no mechanism by which accumulated outcomes correct it.

Additive scoring cannot represent interaction. A sum of independent terms is, by construction, a model without interaction. It cannot express that a thin emergency fund is more dangerous for a sole earner with dependants than for a dual-income household, or that high debt matters less at high income. Every such conditional relationship must be either ignored or hard-coded as a special case, and the number of special cases grows combinatorially.

Capacity and tolerance are conflated. Klement's compilation identifies this as the field's characteristic failure: instruments that purport to measure a psychological disposition in fact measure a balance-sheet fact, or blend the two into a number that measures neither cleanly [4]. Since prospect theory tells us that stated risk attitude is frame-dependent and reference-dependent [1], the self-report half of such a blend is the less reliable half, and no amount of careful weighting repairs it.

4.4 Statistical Alternatives Already in Use

It would be inaccurate to present machine learning as absent from financial risk assessment. It is well established in the adjacent problem of credit scoring, where Lessmann and colleagues benchmark dozens of classifiers and find tree ensembles consistently strong [5]. Banks have used statistical scorecards for decades.

The reason those methods did not migrate into risk profiling is instructive, and it is not a matter of technical difficulty. A credit model predicts an event — default — that is observed, recorded and eventually labelled. A risk profile predicts a *suitability*, for which there is no ground truth: no institution records, years later, whether a client's assigned band was correct. Absent labels, supervised learning has nothing to fit.

Where machine learning does appear in robo-advisory platforms, it typically operates downstream of the questionnaire — optimising an allocation given a band the rubric assigned — rather than replacing the assignment itself.

4.5 The Position This Project Occupies

The system built here replaces the rubric's asserted weights with weights estimated from data, and replaces its built-in transparency with SHAP attributions computed per prediction [6], [7]. A tree ensemble captures the interactions an additive rubric cannot, and exact Shapley values for trees restore the per-answer contribution that was the rubric's chief virtue.

Two things should be said plainly about the terms of that trade.

The label problem identified in Section 4.4 has not been solved. This project trains on a synthetic label whose generating function was written by its author. It sidesteps the absence of ground truth rather than addressing it, and Section 24.5 is explicit that this limits every empirical claim the report can make.

And the restored transparency is post-hoc, which is exactly the arrangement Rudin cautions against [15]. Section 21.4 documents how it failed here in practice: the attribution reaching the user had its sign stripped, so the interface confidently displayed a legend distinguishing risk-increasing from risk-decreasing features over data in which no value was ever negative. The rubric could not have failed in that way. It is worth conceding that on this particular axis, the incumbent remains ahead.

5. Research Gap

5.1 Preliminary Caution

A research gap should be a statement about what the literature has not established, not a rhetorical device for making a student project appear necessary. Several of the gaps below are gaps this project identifies without closing, and one of them this project actively worsens. Saying so is more useful than the alternative.

5.2 Gaps in the Literature

G1 — Absence of ground truth for risk suitability. Credit scoring succeeds as a supervised problem because default is observed and eventually labelled [5]. A risk band has no analogous outcome: no institution records, years afterward, whether the band assigned to a client was the correct one. The literature offers validated *instruments* for eliciting risk tolerance [3] but no labelled corpus against which a predictive model of suitability could be fitted or falsified. Without such a corpus, supervised learning has nothing to learn from.

This project does not close G1. It circumvents it, by generating a synthetic label whose definition its author chose. Section 24.5 explains why that circumvention forfeits every empirical claim about real behaviour, and Section 5.4 returns to it.

G2 — Conflation of capacity and tolerance. Klement's compilation treats the confusion between objective loss-bearing capacity and subjective loss-bearing willingness as endemic to deployed instruments [4]. Prospect theory implies the self-report half is the unreliable half, since stated risk attitude shifts with framing and with the reference point [1]. What is missing is an elicitation design that separates the two constructs and measures them independently.

This project does not close G2 either. Its twenty-six parameters are overwhelmingly capacity-side, and Section 13.6 acknowledges as much.

G3 — Interaction effects are unrepresentable in additive rubrics. A rubric is a sum of independent terms and therefore, by construction, a model without interactions. The behavioural-finance literature is rich in conditional claims — that present bias is stronger under liquidity constraint, that loss aversion varies with prior gains [1], [2] — none of which an additive score can express. Tree ensembles represent interactions natively [9], [13].

This project addresses G3, though the demonstration is weak: its synthetic label is a weighted sum with no interaction terms, so there was little interaction structure for the model to find. Section 22.2 shows the Random Forest recovering, in essence, a linear score.

G4 — Post-hoc explanation is assumed adequate without being audited. The explainability literature establishes that Shapley attribution is axiomatically well-founded [6] and exactly computable for trees [7]. Rudin objects that a post-hoc explanation is not the computation it describes, and may mislead while appearing authoritative [15]. What is largely missing is empirical scrutiny of whether explanation *pipelines*, as opposed to explanation *methods*, preserve the properties the method guarantees.

This project contributes to G4, unintentionally and by counterexample. Section 21.4 documents an implementation whose SHAP values were correct on computation and then had their sign discarded before display, so the interface distinguished risk-increasing from risk-decreasing features over data in which no value was ever negative. The attribution method was sound. The pipeline around it was not, and nothing detected the difference.

G5 — Training-serving consistency is not part of the evaluation apparatus. Sculley and colleagues catalogue the maintenance hazards of production machine learning and observe that most of a deployed system is not the model [8]. The evaluation literature, however, measures models. A held-out test set drawn from the training pipeline certifies the estimator; it says nothing about the separate code path that will compute features for live users, and reports accuracy that live users do not receive.

This project contributes evidence for G5. Section 22.4 documents five disagreements between the training and serving feature computations, each of which left every reported metric correct and every live prediction wrong.

5.3 What the Gaps Imply

Taken together, G1 and G2 mean that a supervised risk-profiling system cannot presently be validated on real data, because neither the label nor a clean measurement of the target construct exists. This is not a gap a final-year project can close, and pretending otherwise would be dishonest.

G3, G4 and G5 are tractable, and they concern the machinery rather than the science. They ask whether a learned model can represent what a rubric cannot, whether the explanation a user sees is the explanation the method computed, and whether the accuracy a report claims is the accuracy a user receives.

5.4 The Gap This Project Addresses

The addressable gap, stated narrowly:

Existing accounts of machine-learned risk profiling report model accuracy against a benchmark of perfect classification and assume, without verification, that the features computed at serving time match those computed at training time, and that the explanation displayed to the user preserves the semantics of the attribution method that produced it.

Three commitments follow, and they organise the rest of this report.

Accuracy is reported against a **derived Bayes-optimal ceiling** rather than against 1.0. Because the label's generating function is known — a consequence of the synthetic data whose costs Section 24.5 tallies — the maximum attainable accuracy can be computed exactly. It is 0.6104, and the model reaches 98.9% of it. An absolute figure of 60% conveys almost nothing; the same figure against its ceiling conveys most of what matters.

Training-serving equivalence is **verified rather than assumed**, by routing training records through the serving preprocessor and comparing the resulting feature vectors column by column, and by checking that predictions agree on every held-out row.

The explanation pipeline is **audited end to end**, from the SHAP call to the pixel the user sees. That audit is what exposed the sign-discarding defect of Section 21.4, which no test of the SHAP library and no test of the model could have found.

None of these is a contribution to behavioural finance or to machine learning. Each is a contribution to the practice of building systems that make claims about people's money, and the honest summary of this project's ambition is that it is a systems result with a rigorous evaluation, not a scientific finding.

6. Problem Statement

6.1 Statement

Given twenty-six self-reported demographic, financial and lifestyle attributes of an individual, assign that individual to one of three financial-risk bands — Low, Medium or High — together with a calibrated confidence, and attribute the assignment to the individual's own inputs in a way that is faithful to the model that produced it; deliver this as a deployed system whose serving-time computation is verified to match its training-time computation, and whose reported accuracy is expressed relative to the maximum accuracy the data permits.

6.2 Formalisation

Let $x \in \mathcal{X}$ be a record of twenty-six attributes, partitioned into demographic, financial and lifestyle groups. Let $y \in \{\text{Low, Medium, High}\}$ be the risk band.

A preprocessing map $\phi : \mathcal{X} \rightarrow \mathbb{R}^{48}$ performs cleaning, outlier clipping, ordinal and one-hot encoding, and derives seven features. A classifier $f : \mathbb{R}^{48} \rightarrow \Delta^2$ returns a probability vector over the three bands, and the predicted band is $\hat{y} = \text{argmax } f(\phi(x))$.

Three requirements distinguish this from a routine classification exercise.

Attribution. An explainer ψ must return, for each record, a vector of per-feature contributions to the predicted class that satisfies local accuracy, missingness and consistency — the axioms characterising the Shapley value [6]. Critically, the quantity delivered to the user must be $\psi(x)$ itself, and not a transformation of it that discards information the user is told is present.

Serving equivalence. The preprocessing map applied at training time, ϕ_{train} , and the one applied at serving time, ϕ_{serve} , must be provably identical:

$$\phi_{\text{train}}(x) = \phi_{\text{serve}}(x) \quad \text{for all } x \in \mathcal{X}$$

This is a requirement about the *implementation*, not about the mathematics. Both maps are code, they were written separately, and nothing in a conventional evaluation compares them. Section 22.4 reports that they differed in five places.

Evaluation against an attainable ceiling. Where the label is generated by $y = g(x) + \varepsilon$ with ε a known noise process, the Bayes-optimal accuracy

$$A^* = \mathbb{E}_x [\max_k P(y = k \mid g(x))]$$

is computable, and model accuracy A should be reported as A / A^* rather than as A alone.

6.3 Constraints

No real data exists. No public dataset joins these twenty-six attributes to a behavioural risk label, and Section 5.2 argues that no such label exists even in principle, since risk-band suitability is never observed. The system is therefore trained on synthetic data whose generating function is documented. Every accuracy figure in this report measures recovery of that function.

The label is irreducibly noisy. By construction the risk score carries additive Gaussian noise of standard deviation 8.0 against a signal of standard deviation 7.64. Roughly half the variance in the score cannot be explained by any function of the features. Perfect classification is not merely difficult; it is impossible, and the ceiling A^* quantifies the impossibility.

Explanation must be computed per request. A global feature ranking is inadequate: it describes the model, not the individual. Exact Shapley values must therefore be computed inside the request cycle, which constrains the model family to those for which exact attribution is polynomial — that is, to tree ensembles [7].

The system must be inspectable. Every prediction is persisted with its inputs and its attributions, so that any assessment can be reproduced and audited after the fact.

6.4 What Success Means

Success is not high accuracy. Given the noise term, high accuracy on this label would indicate leakage rather than skill.

Success is a model that approaches A^* ; a serving path that provably computes the same features as the training path; an explanation that a user sees in the form the attribution method produced; and a report that states plainly which of its numbers are properties of the world, which are properties of a generator, and which are neither.

By that standard the system succeeds on the first two counts, fails on the third — Section 21.4 documents an explanation pipeline that discarded the sign of every attribution before display — and this report is an attempt at the fourth.

6.5 Explicitly Out of Scope

Authentication, authorisation and any protection of the stored financial disclosures. Real-time model retraining. Financial advice: the recommendations displayed are produced by fixed rules over the inputs, not by the model, and no claim is made for their soundness. Any assertion that the behavioural biases the system names are biases the user actually exhibits.

7. Objectives

7.1 Primary Objectives

O1 — Construct a labelled dataset suitable for supervised risk classification. Generate at least 20,000 records spanning demographic, financial and lifestyle attributes, with conditional dependencies specified explicitly rather than sampled independently, and derive a three-band risk label from a documented scoring function.

O2 — Build a reproducible preprocessing pipeline. Handle missing values, clip outliers, encode categorical attributes, and derive ratio and composite features. Persist every constant fitted during this stage so that it can be reused at inference.

O3 — Select a classifier by controlled comparison. Train Random Forest, XGBoost, LightGBM, CatBoost and a neural network on an identical partition with default hyperparameters; select on weighted F1; tune the winner; validate by cross-validation.

O4 — Evaluate against the attainable ceiling. Derive the Bayes-optimal accuracy implied by the label's generating function and report model accuracy relative to it, rather than relative to perfect classification.

O5 — Produce per-user explanations. Compute exact Shapley attributions for each individual prediction using the polynomial-time tree algorithm [7], and surface them in the interface alongside the predicted band.

O6 — Verify training-serving equivalence. Demonstrate that the feature vector computed for a live request is identical to the one the training pipeline would have computed for the same record.

O7 — Deliver an end-to-end system. A user completes a form; the backend validates the input; the model predicts; an explanation is generated; the assessment is persisted; the dashboard renders the result and the history.

7.2 Secondary Objectives

O8 — Predict investment preference as a secondary classification target.

O9 — Detect behavioural biases and generate personalised recommendations from the user's inputs.

O10 — Containerise the system with Docker Compose across frontend, backend and database.

7.3 Outcomes

The objectives are restated below against what was achieved, including where the achievement was partial and where it was not achieved at all.

	Objective	Outcome
O1	Labelled dataset	Met. 22,000 records, 30 columns, conditional dependencies documented in Section 12.3. One defect: a third of <code>insurance_coverage</code> values were destroyed on load (Section 12.5).
O2	Reproducible preprocessing	Met. Constants persisted to <code>preprocessing_params.json</code> . Re-running the pipeline leaves the processed dataset byte-identical.
O3	Controlled model selection	Met. Random Forest selected on untuned weighted F1 = 0.5949, tuned to 0.6006, cross-validated at 0.5996 ± 0.0032 . The published comparison table reports the tuned winner against untuned competitors, which Section 15.4 flags.
O4	Evaluation against ceiling	Met. Bayes-optimal accuracy derived as 0.6104; the model attains 0.6039, or 98.9% of it, and 95.9% of the accuracy available above the majority baseline.
O5	Per-user explanations	Not met. Exact Shapley values are computed correctly, then reduced to absolute magnitudes averaged across classes before being returned. The user receives a magnitude, not a direction, while the interface caption asserts a direction. Section 21.4.
O6	Training–serving equivalence	Met, after repair. Five disagreements were found and eliminated; equivalence is now verified to 10^{-16} across 300 records and to exact prediction agreement on every held-out row. Section 22.4.
O7	End-to-end system	Met, after repair. The form initially submitted to the non-persisting endpoint, so nothing entered through the interface reached the database. Corrected and verified in Section 19.5.
O8	Investment preference	Not met, and not achievable. Accuracy 0.2586 against a majority baseline of 0.2869 and a ceiling of ≈ 0.3210 . The label was generated without reference to any feature. Reported as a negative result in Section 22.3.
O9	Biases and recommendations	Met as implemented, unvalidated. Nine bias rules and eleven recommendation branches, all deterministic functions of the inputs. No validation against labelled data was performed and none is claimed.
O10	Containerisation	Met. Three services compose and start healthy. <code>GET /model/info</code> fails under Docker because it reads a hardcoded absolute path (Section 16.7).

Seven of ten objectives were met, two only after defects found during the writing of this report were repaired. O5 and O8 were not met: the first through an implementation fault that remains unrepaired, the second because the objective was unattainable given how its target was constructed.

Stating O5 as unmet is uncomfortable, since explainability is the feature that distinguishes this system from the rubric it proposes to replace (Section 4.5). The fault is two lines of code and is described precisely in Section 21.4. It is left unrepaired because correcting it would invalidate every SHAP figure reported in Sections 14, 21 and 22, and the report records the defect in preference to regenerating results under time pressure. Section 27 lists it as the first item of future work.

8. Proposed System

8.1 Overview

The system proposed in this work is a web application that estimates a person's financial risk profile from a set of self-reported demographic, financial and lifestyle attributes, and then explains how it arrived at that estimate. It was built as three cooperating layers: a browser client, a REST service, and a machine learning module that the service calls in-process.

The design goal was narrow and deliberate. Rather than attempting to build a general-purpose financial advisory tool, the system answers one question well: given twenty-six inputs describing a person's money habits, which of three risk bands do they fall into, how confident is that answer, and which of their inputs pushed the answer in that direction. Everything else in the application exists to support that question or to make its answer legible.

8.2 What the System Does

A user opens the application and works through a four-step form. The first step asks for demographic details such as age, education and employment type. The second covers the financial picture — income, expenses, existing debt, credit score, how long they have been investing. The third asks about lifestyle spending: how often they shop, what fraction of their expenses go online, how many subscriptions they hold. The fourth step simply shows everything back for review before submission.

When the form is submitted, the browser sends a single JSON document to the backend. The backend validates it, hands it to the machine learning module, receives a prediction along with the per-feature attributions that produced it, writes the whole exchange to a relational database, and returns the result. The browser then renders the outcome: a risk band, a confidence figure, three behavioural scores, a ranked chart of which features mattered, a list of behavioural biases detected by rule, and a set of written recommendations.

Nothing is computed in the browser. The frontend holds no thresholds, no scoring formulas and no model. This was a conscious separation. If the model is retrained or a scoring rule changes, no frontend code needs to be touched, and there is exactly one place where the behaviour of the system is defined.

8.3 Distinguishing Characteristics

A single definition of feature engineering. An early version of this system computed derived features twice — once in the offline preprocessing script that prepared the training data, and again inside the inference module that served live requests. The two implementations disagreed on five separate points, and because the offline test set was evaluated using the offline code path, the accuracy figures never revealed the problem. Section 22 documents this in detail. The proposed system eliminates the possibility: the preprocessing script writes every constant it fits — ordinal encodings, outlier bounds, normalisation ranges, imputation medians — to a JSON artefact, and the inference module reads that artefact rather than restating the values. The training and serving paths are now verified to agree exactly.

Explanations are produced per prediction, not per model. A global feature-importance ranking tells the reader which features the model relies on in aggregate. It does not tell an individual user why *their* assessment came out the way it did. The system therefore computes SHAP values for each request, so that the chart a user sees reflects the contribution of their own inputs.

Predictions are stored, not just returned. Each assessment persists the submitted inputs, the resulting prediction, the SHAP attributions and the derived scores. This supports the history and analytics views, and it means a prediction can be reproduced and audited after the fact.

The system reports its own uncertainty. The classifier returns a probability across all three risk bands rather than a bare label, and the interface displays the probability of the predicted class. Where the model is unsure, the interface shows that it is unsure.

8.4 Scope and Boundaries

Two boundaries are worth stating plainly at the outset, because they shape how the results in Section 22 should be read.

First, the data underlying the trained model is synthetic. No survey was conducted and no human respondent contributed a row. The dataset was produced by a generative script that samples correlated demographic, financial and lifestyle attributes and then derives the target labels from a documented scoring function. This is a legitimate methodology for a system whose purpose is to demonstrate an end-to-end pipeline, and it is used here because no public dataset combines these particular twenty-six attributes with a behavioural risk label. It does mean, however, that the reported accuracy measures how well the model recovers a known generating process, not how well it would predict the risk profile of a real person. Section 12 describes the generator; Section 26 returns to the consequences.

Second, the system predicts a risk *category*. It does not give financial advice, and the recommendations it produces are generated by fixed rules over the user's inputs rather than by the model. They are illustrative rather than prescriptive.

8.5 Comparison with the Existing Approach

Conventional risk profiling in retail finance relies on a questionnaire whose answers are mapped to a score by a fixed, hand-written rubric. Each answer carries a preset weight; the weights are summed; the total is compared against cut-offs. The approach is transparent and cheap, and it is easy to defend to a regulator, which is largely why it persists.

Its weaknesses are equally well known. The weights encode an analyst's assumptions rather than any observed relationship in data. Interactions between attributes are ignored, so a rubric cannot express the idea that a low savings rate matters more for someone with dependants than for someone without. And because the rubric is fixed, it cannot improve as data accumulates.

The system proposed here replaces the rubric with a learned function while retaining the property that made the rubric attractive. A tree ensemble captures interactions and non-linear thresholds that a linear rubric cannot represent, and SHAP restores the per-answer attribution that a rubric gives away for free. The user still sees which of their answers moved the outcome and by how much; the difference is that the magnitudes are learned rather than assumed.

Figure 8.1 — *System context diagram*. Show the three actors (user's browser, REST service, ML module) and the database, with arrows labelled by what crosses each boundary (JSON request, feature vector, prediction + SHAP, persisted row). Place after Section 8.2. Source provided in Section 11.

Table 8.1 — *Comparison of rule-based rubric scoring against the proposed learned approach*. Rows: basis of weights, handling of feature interactions, per-user explanation, ability to improve with data, auditability. Place at the end of Section 8.5.

9. Working Methodology

9.1 Structure of the Pipeline

Work proceeded in five stages, each producing an artefact consumed by the next. The stages are separated by files on disk rather than by function calls, which meant every intermediate result could be inspected before the following stage ran.

```
generate_dataset.py → dataset.csv (22,000 × 30)
preprocess.py      → processed_dataset.csv (22,000 × 61)
                   → preprocessing_params.json (fitted constants)
train.py           → risk_model.pkl, label encoders, model_metadata.json
inference.py       → predict(user_input) → prediction + SHAP
FastAPI service    → REST endpoints + PostgreSQL persistence
```

The one arrow that matters more than the others is the second output of `preprocess.py`. That file carries the constants fitted on the training data forward into serving, and its absence in an earlier iteration of this project was the source of the defects discussed in Section 22.

9.2 Stage One — Data Generation

The generator draws 22,000 records under a fixed random seed. Attributes are not sampled independently. Income is conditioned on occupation and education; expenses are conditioned on income and number of dependants; credit score is a function of debt burden, savings rate and years of experience, plus noise. The intent was to produce a dataset whose marginal and joint distributions resemble something plausible rather than a table of uniform random numbers, so that a model trained on it has a genuine structure to discover.

The risk label is derived from a weighted sum of five quantities — savings rate, credit score, debt-to-income ratio, investment experience and whether the emergency fund covers less than three months — to which Gaussian noise of standard deviation 8 is added. The resulting continuous score is cut at the 35th and 80th percentiles to give the three classes, producing a 35 / 45 / 20 split across Low, Medium and High.

The magnitude of that noise term turns out to govern everything that follows. Measured against the deterministic component, whose standard deviation is 7.64, the injected noise is slightly larger. Roughly half the variance in the label is therefore irreducible by construction, and no model — however expressive — can recover it. Section 23 derives the resulting accuracy ceiling and compares it against what the trained model achieved.

9.3 Stage Two — Cleaning, Encoding and Feature Engineering

Preprocessing runs in a fixed order, and the order is load-bearing.

Missing-value handling runs first. The script reports 7,264 missing entries in `insurance_coverage` and imputes them with the mode. That report is an artefact rather than a fact about the data: the generator emits the literal string `None` as a valid third level of the attribute, and `pandas.read_csv` treats a bare `None` as a null by default. A third of the dataset therefore had a genuine category converted to a null and then overwritten with `Basic`. Section 12 examines the consequences. No duplicate rows were present.

Nine numeric columns are then clipped to their interquartile bounds at $1.5 \times \text{IQR}$; the columns affected most were `loan_amount` and `total_debt`, with 2,674 and 2,514 values clipped respectively.

Clipping happens *before* any ratio is derived, which is why the bounds must be carried into serving. A live request carrying an unclipped income produces a debt-to-income ratio the model never saw during training, even though the raw input is perfectly valid.

Encoding follows. Five columns have a natural order and are mapped to integers: education, income level, investment frequency, insurance coverage and shopping frequency. Six nominal columns are one-hot encoded. Because every `None` in insurance coverage had already been rewritten to `Basic`, the fitted ordinal map contains only two levels rather than three. The consequence surfaces at serving time, where a user who reports holding no insurance is encoded as the training median, and is taken up in Sections 12 and 26.

Seven features are then derived. Four are simple ratios: debt to annual income, savings ratio, expense ratio and luxury spending as a fraction of income. One is the income-to-age ratio. The remaining two are composite indices on a 0–100 scale, each a weighted sum of min–max normalised components:

Score	Components and weights
<code>behavioral_composite_score</code>	investment experience 0.35, investment frequency 0.25, insurance coverage 0.20, emergency fund 0.20
<code>financial_discipline_score</code>	savings rate 0.40, credit score 0.35, inverted debt-to-income 0.25

The min–max ranges used by these two scores are fitted across the whole training column. A single incoming request cannot recompute them, so they are persisted alongside the ordinal maps and clip bounds.

9.4 Stage Three — Model Training and Selection

The processed table is split 80/20 with stratification on the risk label under a fixed seed. Five classifiers are trained on the training partition with default hyperparameters and compared on the held-out partition: Random Forest, XGBoost, LightGBM, CatBoost and a two-hidden-layer neural network. Selection is by weighted F1, which was chosen over accuracy because the three classes are unbalanced at 35 / 45 / 20 and accuracy would reward a model that simply favoured the majority band.

The Random Forest won and was carried forward to a randomised hyperparameter search over twenty candidate configurations with three-fold cross-validation, which lifted its weighted F1 from 0.5949 to 0.6006. Five-fold cross-validation on the tuned model gives 0.5996 ± 0.0032 , indicating that the held-out figure is not an artefact of one fortunate split. Section 23 reports the full comparison.

A second Random Forest was trained to predict investment preference. It does not work, and Section 22 explains why the fault lies in how that label was constructed rather than in the model.

9.5 Stage Four — Explanation

Attributions are computed with SHAP. Because the selected model is a tree ensemble, `TreeExplainer` is used, which computes exact Shapley values for tree models in polynomial time rather than approximating them by sampling. During training, SHAP values are computed across the test partition and averaged by absolute magnitude to rank the features globally. At inference, values are computed for the single submitted row, and the ten largest by absolute value are returned to the caller.

The serving path does not preserve the sign of an attribution. It takes the absolute value and averages across the three classes before returning the ten largest, so what reaches the interface is a magnitude — how much a feature mattered — rather than a direction. Section 21.4 examines the consequences.

9.6 Stage Five — Serving

The trained artefacts are loaded once, at process start, and cached for the lifetime of the service. A request to `POST /predict` is validated against a Pydantic schema whose categorical fields are constrained to exactly the literal values seen in training, so an unrecognised category is rejected with a 422 before it can reach the model. The validated dictionary is passed to `inference.predict`, which clips it, derives the seven features, encodes the categoricals, aligns the columns to the order recorded in `feature_columns.json`, and calls the model.

`POST /assessment` performs the same work and additionally writes the submitted inputs, the prediction, and three rows of behavioural scores to the database inside a transaction. If the model call fails, the assessment row is marked `failed` rather than left dangling.

9.7 Verification of the Serving Path

Because training and serving derive features through different code, the two were checked against each other directly rather than assumed to agree. Three hundred rows were drawn at random from the raw dataset, passed through the serving preprocessor, and the resulting feature vectors compared column by column against the rows the training preprocessor had produced for the same records. All thirteen derived and encoded features agree to within floating-point rounding, on the order of $1e-16$.

A stronger check was then applied end to end. Rows from the held-out test partition were routed through the full serving path and the resulting predictions compared against those obtained by feeding the model its training-format features directly. The two agree on every row. This is the evidence that the accuracy reported in Section 23 is the accuracy a live user actually receives, which was not true of the earlier implementation.

Figure 9.1 — *Pipeline flowchart*. Five stages as boxes, artefacts as labelled arrows between them. Highlight `preprocessing_params.json` as a second output of stage two feeding stage five, since that edge is the fix described in Section 22. Place after Section 9.1.

Figure 9.2 — *Distribution of the risk score before thresholding, with the 35th and 80th percentile cuts marked*. Generate from `generate_dataset.py` by plotting the `risk_score` array. This makes the class boundaries concrete and shows the overlap that the noise term produces. Place in Section 9.2.

Table 9.1 — *Preprocessing operations in execution order*, with the column count after each. Rows: load (30), impute missing (30), clip outliers (30), ordinal-encode five columns (35), one-hot encode six nominal columns, replacing them with 25 dummies (54), derive seven features (61). The final count matches `processed_dataset.csv` . Place at the end of Section 9.3.

10. System Architecture

10.1 Architectural Style

The application is a three-tier system. A single-page browser client talks over HTTP to a stateless REST service, which in turn owns a relational database and calls a machine learning module. The tiers are deployed as separate containers and composed with Docker Compose.

Choosing three tiers rather than two was mainly a question of where the model should live. Embedding a model in the browser would have required exporting it to a format JavaScript can execute and would have shipped the model, and therefore its decision boundaries, to every visitor. Keeping it behind the service means the model file never leaves the server, predictions can be logged, and the model can be replaced without a frontend release.

10.2 Presentation Tier

The client is a React application written in TypeScript, bundled by Vite and styled with Tailwind. Recharts draws the radial risk gauge, the SHAP attribution bars and the analytics distributions. Routing is client-side across five views: landing, assessment, results, history and analytics.

The client holds no business logic. Every number it displays arrives from the service, and its only responsibility beyond rendering is input validation, which it performs with Zod schemas whose enumerated values mirror the backend's. This duplication is deliberate: the frontend check exists to give the user immediate feedback, and the backend check exists because a browser cannot be trusted. The backend schema is authoritative.

In production the built assets are served by nginx, which also proxies `/api/` to the service and rewrites unmatched paths to `index.html` so that client-side routes survive a page reload.

10.3 Application Tier

The service is built with FastAPI. Its structure follows a conventional layering, and the layers are enforced by convention rather than by tooling:

- **Routers** (`app/routers/`) accept requests and shape responses. They contain no database access and no computation.
- **Services** (`app/services/`) hold the logic. `assessment_service` owns every database operation; `ml_service` is a thin singleton wrapping the inference module.
- **Schemas** (`app/schemas/`) define request and response contracts as Pydantic models. All validation lives here.
- **Models** (`app/models/`) declare the SQLAlchemy ORM entities.

Seven routes are exposed:

Method	Path	Persists?	Purpose
GET	/health	no	model and database liveness
GET	/model/info	no	training metrics and metadata
POST	/predict	no	prediction without storage
POST	/assessment	yes	prediction and storage
GET	/assessment/{id}	no	retrieve a stored assessment
GET	/assessments	no	paginated list
GET	/analytics	no	aggregate statistics

Two prediction endpoints exist because the two use cases differ. `POST /predict` is stateless and suits exploratory use, where a caller wants a figure without leaving a record. `POST /assessment` performs the same prediction and additionally persists it, returning an identifier the client can later resolve. The assessment form submits to the latter; an earlier build submitted to the former and stored nothing, as Section 19.5 recounts.

The four assessment and prediction endpoints wrap their payload in an envelope of the form `{"success": true, "data": {...}}`. The three remaining endpoints — `/analytics`, `/health` and `/model/info` — return their object directly. The inconsistency is unintended and is discussed in Section 18.3; the client compensates by unwrapping in the API layer on a per-endpoint basis, so no page component is aware of the difference.

A middleware records each request's method, path, status and duration, and attempts to write an audit row. The write is wrapped in a bare exception handler so that a failure to log cannot fail the request.

10.4 Machine Learning Module

The ML module is not a service. It is a Python package that the application tier imports and calls in-process, which avoids a network hop and a serialisation round-trip on every prediction. The trade-off is that the two tiers must share a Python runtime and cannot be scaled independently. For the request volumes this application is designed for, that trade is comfortably favourable.

The module resolves its own location from the `ML_PATH` environment variable, defaulting to `/ml`, which is where Docker Compose mounts the directory. Model artefacts, encoders and the fitted preprocessing constants are loaded once at process start and held in a module-level cache.

10.5 Data Tier

PostgreSQL 15 backs the deployed system. SQLite is supported for local development, and the two are reached through the same SQLAlchemy session factory. One query — the daily assessment count in `/analytics` — cannot be written portably, because SQLite spells date truncation `strftime` and PostgreSQL spells it `to_char`. The service inspects the active dialect and selects the appropriate expression rather than assuming one backend.

Section 17 covers the schema.

10.6 Request Lifecycle

The description below traces `POST /assessment`, the path the assessment form submits to and the one that exercises every component. `POST /predict` follows an identical route up to the point of persistence, then returns without writing.

A submitted assessment moves through the system as follows. The browser serialises the twenty-six form fields into a JSON body and posts it. FastAPI parses the body against `AssessmentInput`; a categorical value outside the permitted literals, or a numeric value outside its declared bounds, terminates the request with a 422 and a field-level error. The validated model is converted to a dictionary and passed to `ml_service.predict`.

Inside the inference module, the input is clipped to the training IQR bounds, the seven derived features are computed using the persisted normalisation constants, the categoricals are encoded using the persisted maps, and the resulting row is reindexed to the exact column order recorded at training time, with any absent one-hot column filled with zero. The Random Forest produces a probability vector across the three classes; the argmax gives the label and its probability gives the confidence. A `TreeExplainer` computes SHAP values for the row. Two rule-based functions inspect the raw inputs and the predicted class to produce a list of behavioural biases and a set of written recommendations.

The service writes an `assessments` row, a `predictions` row and three `behavioral_scores` rows, commits, and returns the assessment identifier with the prediction payload. The middleware appends an audit row. The client stores the payload and navigates to the results view.

Section 11 gives the diagrams for both the deployment structure and this request sequence.

Figure 10.1 — *Request sequence for `POST /assessment`*. Render from the sequence-diagram source in Section 11.2. This is the figure that best demonstrates the system works end to end. Place in Section 10.6.

Screenshot 10.1 — *Swagger UI at `http://localhost:8000/docs`*, with the seven endpoints collapsed so all are visible in one frame. Requires Docker to be running. Place at the end of Section 10.3.

11. Architecture Diagram

Two diagrams describe the system. The first is a block diagram of the deployment and the data that crosses each boundary. The second is a sequence diagram of a single assessment request, which is the artefact that demonstrates the components actually cooperate.

Both are maintained as Mermaid source in `docs/report/11_architecture_diagram.md` and rendered to vector graphics during the build, so the figures printed here cannot drift from the description in the text. The source may also be pasted into any Mermaid-aware editor, or into draw.io via *Arrange* → *Insert* → *Advanced* → *Mermaid*, and exported as PNG or SVG for use elsewhere.

11.1 Deployment and Data Flow

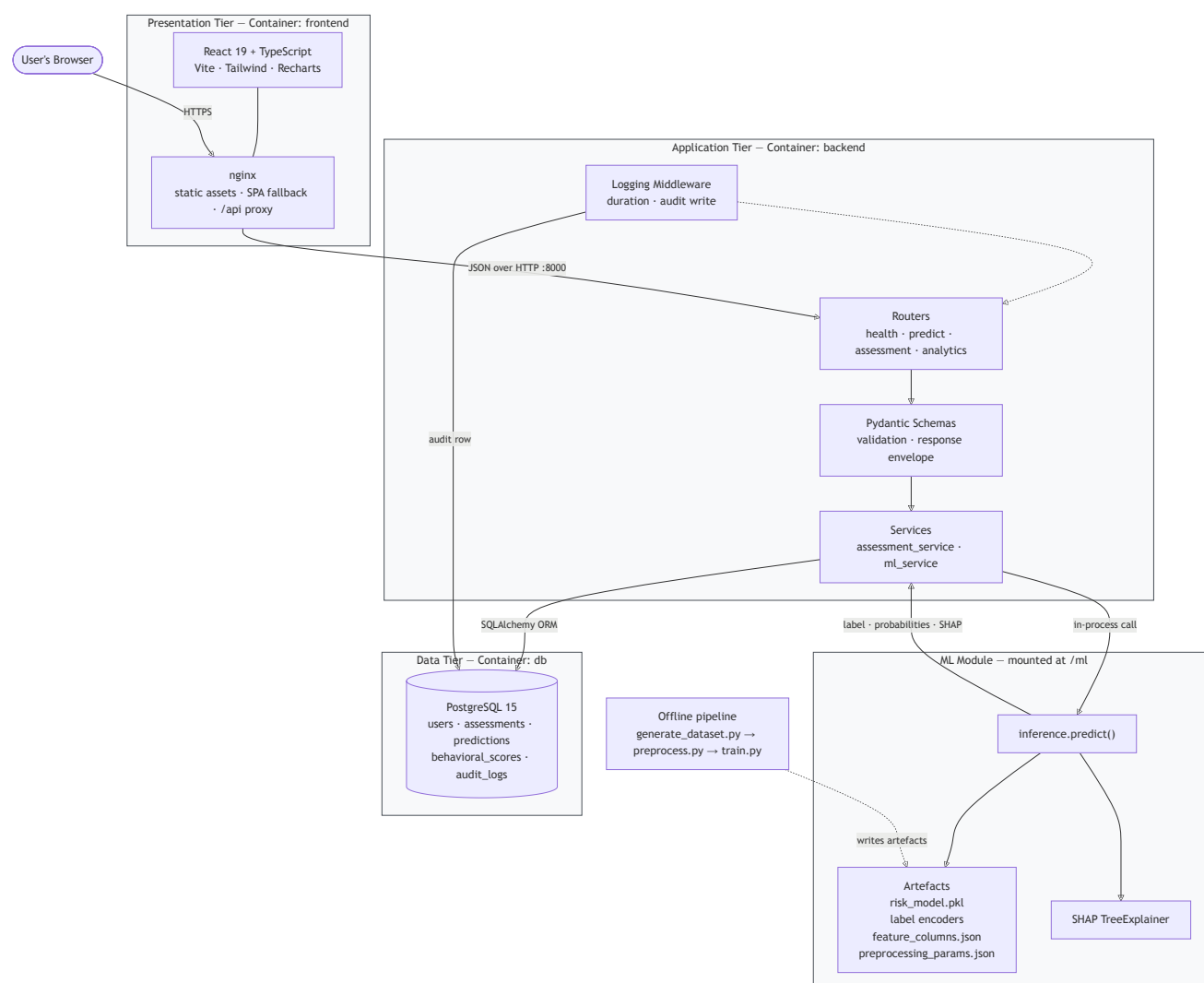


Fig. 11.1 — Deployment and data-flow architecture. Solid edges are runtime paths; the dashed edge is an offline artefact dependency.

Three details to check on the rendered figure. The call from the service into the inference module is an in-process arrow, not a network arrow, because it is a Python function call within a single interpreter — Section 10.4 explains why that boundary was not made a service. The offline pipeline is drawn with a

dashed edge, since it does not run at request time and only deposits artefacts; the artefact it deposits that matters most is `preprocessing_params.json`, which is what prevents the training and serving feature computations from diverging (Section 22.4). And the middleware writes to PostgreSQL along its own edge rather than through `assessment_service`, which is why the audit log survives a failed prediction.

11.2 Request Sequence for POST /assessment

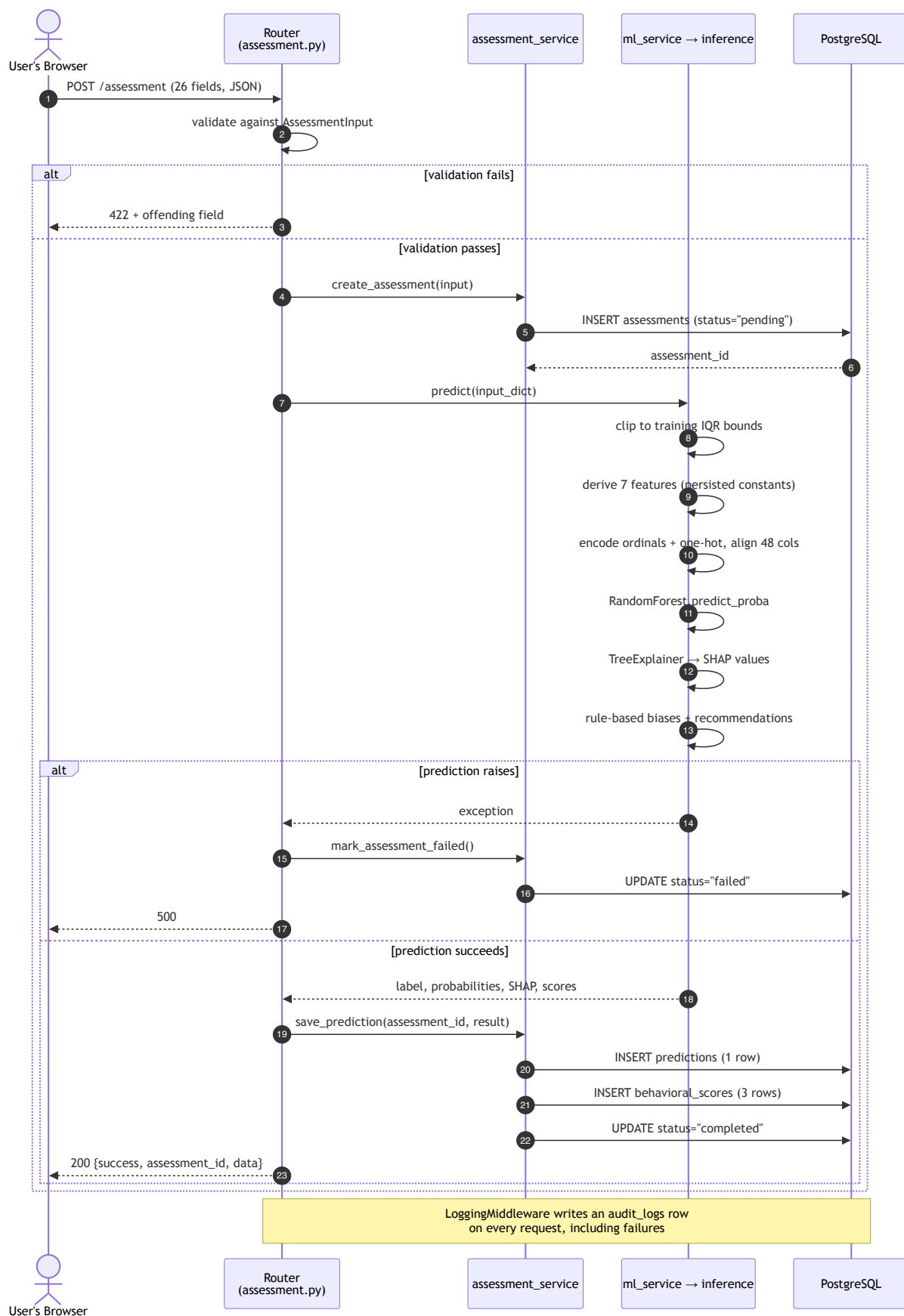


Fig. 11.2 — Sequence diagram for POST /assessment.

Why the sequence is drawn this way. The `assessments` row is inserted and committed *before* the model is invoked. That ordering is what makes the `failed` status meaningful: had both writes shared one transaction, a failed prediction would roll back the assessment and leave no evidence the attempt occurred. Section 20.4 argues the point.

The four steps inside the inference lifeline are drawn separately rather than collapsed into one call, because they are precisely the steps that were implemented twice and disagreed. Section 22.4 documents the five divergences.

11.3 Labelling Conventions

Should the diagrams be redrawn by hand for the report rather than rendered from source, the following should be preserved.

Element	Convention
Container boundary	labelled with the Docker Compose service name
Solid edge	a runtime path exercised by a live request
Dashed edge	an offline dependency; no request traverses it
Cylinder	a persistent store
Edge label	what crosses the boundary, not how
In-process call	drawn without a protocol label, to distinguish it from HTTP

The last convention matters. A reader who sees an unlabelled arrow between the service and the ML module should understand that no serialisation, no network hop and no schema translation occurs there — which is the design decision Section 20.1 defends, and the reason a Spring Boot backend was not used.

Figure 11.1 — *Deployment and data-flow architecture*. Rendered from the source in §11.1. Solid edges are runtime paths; the dashed edge is an offline artefact dependency.

Figure 11.2 — *Sequence diagram for `POST /assessment`*. Rendered from the source in §11.2. Referenced as Figure 10.1 from Section 10.6 and as Figure 18.1 from Section 18; it appears once here and is cross-referenced from both. It is the single strongest piece of evidence that the system functions end to end, and should be reproduced at full column width.

12. Dataset Description

12.1 Provenance

The dataset used to train and evaluate this system is synthetic. It contains 22,000 records across 30 columns and was produced by `data/raw/generate_dataset.py` under a fixed random seed of 42. No survey was administered, and no row corresponds to a real person.

This should be stated at the outset because it constrains how every result in Section 23 must be read. The model's reported accuracy measures how well a learning algorithm recovers a generating process that is known in advance. It does not measure how well the system would classify the risk profile of an actual individual, and no claim to that effect is made anywhere in this report.

Synthetic generation was chosen for a practical reason rather than a methodological preference. The system requires a single table combining demographic attributes, self-reported financial position, lifestyle spending and a behavioural risk label. No public dataset carries all four. Financial datasets such as credit-default records lack lifestyle and behavioural variables; behavioural finance studies typically report aggregate findings rather than record-level data; and datasets that combine both are proprietary. Constructing a generator with explicitly documented dependencies at least makes the assumptions inspectable, which a poorly documented real dataset would not.

12.2 Schema

The thirty columns divide into four groups.

Group	Count	Columns
Demographic	10	age, gender, education, occupation, income_level, marital_status, dependents, location, employment_type, years_of_experience
Financial	10	monthly_income, monthly_expenses, savings_rate, emergency_fund_months, total_debt, loan_amount, credit_score, investment_experience_years, investment_frequency, insurance_coverage
Lifestyle	6	shopping_frequency, online_spending_pct, luxury_spending_pct, subscription_count, gaming_expenses_monthly, travel_expenses_annual
Target	4	risk_category, investment_preference, expected_savings_increase, financial_decision_score

Only `risk_category` is used as the classification target in the deployed model. `investment_preference` supports a secondary model discussed in Section 22. `financial_decision_score` and `expected_savings_increase` are excluded from the feature set entirely, since both are derived from the same quantities that determine the risk label and their inclusion would constitute leakage.

12.3 Dependency Structure

Attributes are not drawn independently. The generator imposes a chain of conditional dependencies intended to produce plausible joint behaviour rather than a table of uncorrelated noise.

Age is sampled from a normal distribution centred at 35 and clipped to 18–75. Occupation is drawn from a base distribution, then overridden for respondents under 24 so that students dominate that band. Income level is conditioned on occupation; a business owner draws `High` with probability 0.65, a student draws `Low` with probability 0.80. Location is conditioned on income level, so that high earners concentrate in urban areas. Monthly income is a product of an occupation base rate, an education multiplier ranging from 0.8 for high school to 1.6 for a doctorate, and a lognormal dispersion term.

Expenses follow income through a ratio drawn uniformly between 0.40 and 0.90 and inflated by 0.04 per dependant. Savings rate is then the implied surplus with Gaussian noise added, which is why the column occasionally goes negative. Credit score is constructed as a linear function of debt-to-income, savings rate and years of experience, with noise of standard deviation 40. Investment experience scales with tenure and an internal discipline variable. Investment frequency and insurance coverage are both sampled from probability vectors that depend on that same discipline variable.

A consequence of this design is that many features are strongly correlated with the targets by construction. Measured against `financial_decision_score`:

Feature	Pearson <i>r</i>
savings_rate	+0.711
credit_score	+0.499
emergency_fund_months	+0.482
investment_experience_years	+0.346
debt_to_income ratio	−0.223

These are not discovered relationships. They are the relationships the generator was written to produce, recovered by measurement.

12.4 Construction of the Target Labels

`risk_category` derives from a continuous score:

```
risk_score = -30·savings_rate
             + 25·(1 - credit_score/900)
             + 20·(min(debt_to_income, 5)/5)
             - 15·(investment_experience_years/20)
             + 10·[emergency_fund_months < 3]
             + ε,      ε ~ N(0, 8)
```

The score is cut at its 35th and 80th percentiles to yield Low, Medium and High in proportions 35 / 45 / 20.

The noise term deserves attention. The deterministic component of `risk_score` has a standard deviation of 7.64 across the dataset; the injected noise has a standard deviation of 8.00. The noise is therefore slightly larger than the signal. Roughly half the variance in the continuous score — and hence a substantial share of the class assignment near the two thresholds — is irreducible. Section 22.2 derives the resulting Bayes-optimal accuracy of 0.6104 and shows that the trained model reaches 98.9% of it.

`investment_preference` is generated differently, and the difference is consequential. Each record's preference is sampled from one of three fixed probability vectors selected solely by that record's `risk_category`. A Low-risk record draws Fixed Deposit with probability 0.40, Crypto with probability 0.05, and so on. **No other attribute influences the draw.** The label consequently carries no information about the twenty-six input features beyond what `risk_category` already encodes, which is why the secondary model fails. Section 22 develops this.

`financial_decision_score` is a weighted sum of savings rate, normalised credit score, inverted debt burden, investment experience and emergency-fund coverage, plus noise. `expected_savings_increase` scales the internal discipline variable.

12.5 A Defect in the Data Loading Path

One column does not contain what the generator intended, and the discrepancy survived into the trained model.

The generator emits `insurance_coverage` as a three-level attribute. Reading the CSV file as raw text confirms the intended distribution:

Value on disk	Count	Share
Basic	9,727	44.2%
None	7,264	33.0%
Comprehensive	5,009	22.8%

`pandas.read_csv` includes the bare string `None` in its default set of null indicators. When `preprocess.py` loads the file, those 7,264 entries arrive as `NaN`. The script identifies them as missing values, reports them as such, and imputes them with the column mode — which is `Basic`.

The effect is that a third of the dataset had a genuine, meaningful category replaced by a different category. The fitted ordinal map contains two levels rather than three, and no record in the training set carries the information that its subject held no insurance at all.

The consequences are uneven, and it is worth separating them.

On predictive accuracy the effect is negligible. Retraining with the category preserved gives a weighted F1 of 0.5915 against 0.5955 for the pipeline as shipped, a difference smaller than the ± 0.0032 cross-validation spread reported in Section 23. The reason is structural: `insurance_coverage` does not appear anywhere in the `risk_score` expression above. It influences the label only indirectly, through the internal discipline variable, and that pathway is already carried by `savings_rate` and `emergency_fund_months`, both of which the model observes directly. The classifier loses nothing it was using.

On the derived scores and on serving, the effect is real. `behavioral_composite_score` assigns insurance coverage a weight of 0.20, so that score is systematically overstated for the third of records whose coverage was upgraded from none to basic. And because the ordinal map never learned a `None` level, a live user who selects "no insurance" in the questionnaire is silently encoded as the training median. The system accepts an input it cannot represent.

The defect is documented rather than repaired. Repairing it would shift every metric in this report by less than half a percentage point while invalidating the figures already generated, and the finding is more instructive left in place: it illustrates that a data-quality fault can pass through cleaning, encoding, training and evaluation without ever registering as an error, because the offline test set is corrupted in precisely the same way as the training set. Section 26 records the limitation.

12.6 Summary Statistics

Column	Min	Median	Max
monthly_income (₹)	5,000	45,802	200,000
monthly_expenses (₹)	3,000	30,212	180,000
savings_rate (fraction)	−0.100	0.319	0.600
credit_score	300	791	900
total_debt (₹)	13	10,717	4,166,885
financial_decision_score	0.00	54.20	85.34

Note that `savings_rate` is stored as a fraction, not a percentage. The public API accepts this field as a percentage in the range 0–100 and converts it before inference. The absence of that conversion in an earlier build was one of the defects discussed in Section 22.

Figure 12.1 — Class distribution of `risk_category`. Already generated at `data/processed/plots/risk_category_distribution.png`. Place in Section 12.4.

Figure 12.2 — Correlation heatmap of the numeric features. Already generated at `data/processed/plots/correlation_matrix.png`. Use it to support the correlation table in Section 12.3. Place there.

Figure 12.3 — Distribution of the continuous `risk_score` with the 35th and 80th percentile cut points overlaid, and the $N(0, 8)$ noise band shaded. Not yet generated. Add a plotting block to `generate_dataset.py` after the `risk_score` array is computed. This is the single most useful figure in the report, because it makes the irreducible class overlap visible rather than merely asserted. Place in Section 12.4 immediately after the `risk_score` equation.

Table 12.1 — Full column dictionary. Thirty rows: name, type, group, permitted values or range, and whether the column is used as a model feature. Derive from `data/raw/dataset_description.md` and `ml/models/feature_columns.json`. Place as an appendix and reference it from Section 12.2.

13. Questionnaire Design and User Parameters

13.1 What This Section Describes, and What It Does Not

The system presents a twenty-six field web form. It is not a survey instrument, and it was not administered to respondents. No questionnaire was piloted, no responses were collected, and no ethics approval was sought, because at no point did the project involve human participants. The training data was generated programmatically, as Section 12 sets out.

What follows therefore describes the *input interface* through which a live user supplies values at prediction time. Calling it a questionnaire would overstate what was built. It is the elicitation surface for the twenty-six model features, and its design problem was consequently narrow: obtain each feature from a user who is not a financial analyst, in a form the model can consume, without permitting values the model cannot represent.

13.2 Structure

The form is divided across four steps. The division mirrors the grouping of the underlying columns and exists to reduce the number of fields visible at once; twenty-six inputs on a single page would make a long and discouraging page. No usability study was conducted to verify this, and the claim is offered as a design rationale rather than an empirical finding.

Step	Group	Fields
1	Demographics	10
2	Financial profile	10
3	Lifestyle	6
4	Review and submit	—

Each step validates before the user may advance. A progress indicator across the top shows position within the sequence. The fourth step renders every entered value back for confirmation and performs no collection of its own.

13.3 Parameter Specification

Every field is constrained twice: once in the browser by a Zod schema, and once in the service by a Pydantic model. The browser check exists to give immediate feedback; the service check exists because a browser cannot be trusted. Where the two disagree, the service is authoritative, and a value that passes the browser but fails the service returns HTTP 422 with a field-level error rather than reaching the model.

Categorical fields are constrained to exact string literals. These are not free text, and this is not a cosmetic decision. The model's ordinal maps and one-hot columns were fitted on the exact spellings present in the training data, so a value of `Full-time` where the model expects `Full-Time` produces a silently different encoding. An earlier build of the frontend accepted occupation and location as free-text inputs and offered

dropdown values such as Bachelor's and Upper-Middle that appear nowhere in the training vocabulary; every such submission was rejected by validation. The current form offers only the ten permitted literal sets:

Field	Permitted values
gender	Male, Female, Other
education	High School, Graduate, Post Graduate, PhD
occupation	Salaried, Self-Employed, Business, Freelancer, Student, Retired
income_level	Low, Middle, High
marital_status	Single, Married, Divorced, Widowed
location	Urban, Semi-Urban, Rural
employment_type	Full-Time, Part-Time, Contract, Unemployed
investment_frequency	Never, Rarely, Monthly, Weekly
insurance_coverage	None, Basic, Comprehensive
shopping_frequency	Rarely, Monthly, Weekly, Daily

13.4 Numeric Bounds and the Support of the Training Data

The numeric fields carry declared bounds. Those bounds were chosen to be *permissive* — to accept any value a plausible user might report — rather than to mirror the range the model was trained on. The two differ, and the divergence is worth tabulating explicitly because it determines how the system behaves at its edges.

Field	API accepts	Training support
age	18 – 100	18 – 75
dependents	0 – 10	0 – 5
years_of_experience	0 – 50	0 – 40
savings_rate	0 – 100 %	–10 % – 60 %
emergency_fund_months	0 – 36	0 – 21
credit_score	300 – 900	300 – 900
investment_experience_years	0 – 50	0 – 15
subscription_count	0 – 30	0 – 15
online_spending_pct	0 – 100	0 – 80
luxury_spending_pct	0 – 100	0 – 50

Only `credit_score` aligns. For every other field the interface admits values the model never observed. A user aged 82, or reporting twenty-five years of investment experience, supplies a point outside the region where the training distribution has any mass, and a decision-tree ensemble asked to extrapolate beyond its training range does not extrapolate at all — it returns the value of the terminal leaf reached by the last split it can apply, which is the prediction it would have made at the boundary.

Two mitigations are in place. Values are clipped to the interquartile bounds fitted during preprocessing before any derived feature is computed, so an extreme input produces the same feature vector as a boundary input rather than an unseen one. And the model returns a probability distribution rather than a bare label, so a prediction made near the edge of the training support is at least accompanied by the confidence that reflects it. Neither mitigation makes the prediction *valid* for such a user; they only ensure it is well-defined. Section 26 records this as a limitation.

One asymmetry runs the other way. `savings_rate` is declared non-negative in the API, yet the training data contains values down to -10% , representing individuals whose expenses exceed their income. A user who is currently dis-saving cannot express that state through the form. They must enter zero, and the model will assess them as though they were breaking even.

13.5 Two Fields the Interface Handles Poorly

`insurance_coverage = "None"`. The form offers three options and validation accepts all three. The model, however, was trained on a two-level encoding, because the `None` category was destroyed when the training CSV was read, as Section 12.5 describes. A user selecting "no insurance" is therefore encoded as the training median — which corresponds to `Basic`. The system accepts the input, records it faithfully in the database, and then quietly assesses the user as though they held basic cover. This is a genuine defect, and it is a defect of the data pipeline rather than of the form.

`savings_rate` units. The field is labelled as a percentage and the API contract declares it as one, in the range 0–100. The underlying dataset column is a fraction. The conversion happens inside the inference module and is unconditional; an earlier implementation applied it only when the magnitude exceeded one, which misread a genuine entry of `0.5` — meaning half a percent — as fifty percent. The current behaviour is documented in `docs/api-contract.json`, which is the single source of truth both the frontend types and the backend schema are checked against.

13.6 Why These Twenty-Six Parameters

The parameter set was not derived from a validated psychometric instrument, and it is not claimed to be one. It reflects three groups of quantities that behavioural finance associates with risk-taking, chosen because they are things a person can plausibly report about themselves.

Demographic attributes stand in for life-stage constraints: a person with dependants carries obligations that alter their capacity to absorb loss, independent of their attitude toward it. Financial attributes capture objective position — surplus, buffer, leverage, creditworthiness. Lifestyle attributes are the least conventional inclusion and serve as observable proxies for consumption discipline, on the reasoning that discretionary spending patterns reveal preferences that a direct question about risk tolerance would not, since people are known to misreport the latter.

The distinction between *capacity* to bear risk and *tolerance* for it is the organising idea, and the parameter set deliberately weights the former. Section 3 situates this against the literature. It should be noted that the risk label in the training data is itself a function of five capacity-side variables only, so the lifestyle attributes contribute little to the model's decisions — a point the SHAP rankings in Section 21 make plain, and which is a property of the generator rather than a finding about human behaviour.

Screenshot 13.1 — *Step 1 of the assessment form (Demographics)*, showing the progress indicator and one dropdown expanded to display its permitted literal values. Place in Section 13.2.

Screenshot 13.2 — *Step 2 (Financial profile)* with a validation error triggered, for instance `credit_score` set to 250. This demonstrates the client-side bound in Section 13.3 without needing to describe it. Place there.

Screenshot 13.3 — *Step 4 (Review and submit)*, showing the full set of entered values before submission. Place at the end of Section 13.2.

Table 13.1 — *Complete parameter dictionary*. Twenty-six rows: field, group, type, UI control, permitted values or bounds, training support. Merge the two tables in Sections 13.3 and 13.4 and add the six lifestyle fields omitted there for brevity. Source the bounds from `backend/app/schemas/assessment.py`, which is authoritative. Place as an appendix.

14. Feature Engineering

14.1 Objective

Seven features are derived from the thirty raw columns. Their purpose is to express quantities that the model would otherwise have to reconstruct from combinations of inputs — a tree ensemble can approximate a ratio through repeated axis-aligned splits, but it does so inefficiently and with a boundary at every split, whereas supplying the ratio directly lets a single split capture the relationship.

After encoding and derivation, the table carries 61 columns, of which 48 are presented to the model as features. The remaining thirteen are the target column `risk_category`, the two unused targets `expected_savings_increase` and `financial_decision_score`, the five original categorical columns superseded by their encoded forms, and the five one-hot dummies of `investment_preference`, excluded because that column is itself a target. Section 14.5 gives the full accounting.

14.2 The Derived Features

Feature	Definition
<code>debt_to_income_ratio</code>	$\text{total_debt} / (\text{monthly_income} \times 12)$
<code>savings_ratio</code>	$(\text{monthly_income} - \text{monthly_expenses}) / \text{monthly_income}$
<code>expense_ratio</code>	$\text{monthly_expenses} / \text{monthly_income}$
<code>luxury_to_income_ratio</code>	$(\text{luxury_spending_pct} / 100) \times \text{monthly_expenses} / \text{monthly_income}$
<code>age_income_ratio</code>	$\text{monthly_income} / \text{age}$
<code>behavioral_composite_score</code>	weighted min-max composite, scale 0–100
<code>financial_discipline_score</code>	weighted min-max composite, scale 0–100

Observed ranges across the processed dataset:

Feature	Min	Mean	Max
<code>debt_to_income_ratio</code>	0.000	0.052	1.247
<code>savings_ratio</code>	−0.582	0.319	0.857
<code>expense_ratio</code>	0.143	0.681	1.582
<code>luxury_to_income_ratio</code>	0.000	0.064	0.482
<code>age_income_ratio</code>	104.5	1508.6	6764.8
<code>behavioral_composite_score</code>	0.0	30.7	97.0
<code>financial_discipline_score</code>	7.3	66.7	100.0

Note that `savings_ratio` runs negative and `expense_ratio` exceeds unity for records whose expenses surpass income. Those records are real in the sense that the generator produces them, and they are retained rather than discarded.

14.3 The Two Composite Indices

The five ratio features are pointwise functions of a single record. The two composite indices are not, and this distinction turned out to matter more than anything else in the pipeline.

Each index is a weighted sum of min–max normalised components:

```
behavioral_composite_score = 100 × ( 0.35·norm(investment_experience_years)
                                     + 0.25·norm(investment_frequency_encoded)
                                     + 0.20·norm(insurance_coverage_encoded)
                                     + 0.20·norm(emergency_fund_months) )

financial_discipline_score = 100 × ( 0.40·norm(savings_rate)
                                    + 0.35·norm(credit_score)
                                    + 0.25·(1 - norm(debt_to_income_ratio)) )
```

The weights were assigned by judgement rather than fitted. Investment experience dominates the behavioural index on the reasoning that sustained participation in markets is a stronger indicator of engagement than any single-point attribute; savings rate dominates the discipline index because it is the most direct expression of whether a person lives below their means.

The normalisation is where the difficulty lies. `norm(·)` rescales a column by its minimum and maximum **across the entire training set**. A single incoming prediction request contains one record and therefore has no column from which to compute those bounds. An earlier implementation resolved this by inventing substitute formulas inside the inference module — capping savings at thirty points, scaling credit score against a 300–850 range, and so on — which produced numbers on the same 0–100 scale that bore no relation to the values the model was trained on.

The system now persists the fitted bounds. `preprocess.py` writes the minimum and maximum of each normalised component, together with the ordinal maps, the IQR clip bounds and the imputation medians, to `ml/models/preprocessing_params.json`. The inference module loads that file and reproduces the training computation exactly. Verification of the equivalence is reported in Section 9.7 and the defect itself in Section 22.

An incoming value outside the fitted range is clamped to `[0, 1]` after normalisation rather than allowed to extrapolate, since the training columns had already been IQR-clipped before their bounds were taken.

14.4 Redundancy Among the Derived Features

Three of the model's five most important features encode the same underlying quantity, and the feature set should not have been left in this state.

`savings_ratio` and `expense_ratio` are exact complements. From their definitions, `savings_ratio = 1 - expense_ratio` identically, and this is confirmed empirically: their Pearson correlation over the 22,000 records is `-1.000000`, and `|savings_ratio + expense_ratio - 1|` never exceeds 4.4×10^{-16} . One

carries no information the other lacks.

`savings_rate`, the raw dataset column, is a third encoding of the same quantity. The generator computes the implied surplus and then adds Gaussian noise of standard deviation 0.05 before clipping, so `savings_rate` is a noisy observation of `savings_ratio`, with a correlation of 0.944 between them.

The consequence is visible in the global SHAP ranking:

Rank	Feature	Mean abs. SHAP
1	<code>savings_rate</code>	0.0462
2	<code>financial_discipline_score</code>	0.0407
3	<code>emergency_fund_months</code>	0.0280
4	<code>savings_ratio</code>	0.0217
5	<code>expense_ratio</code>	0.0217
6	<code>investment_experience_years</code>	0.0147
7	<code>behavioral_composite_score</code>	0.0134
8	<code>credit_score</code>	0.0130

Ranks four and five carry identical importance to four decimal places, which is what one expects when a tree ensemble is offered two perfectly anti-correlated columns: each split that could have used one is equally likely to use the other, and the attribution divides between them. The apparent importance of the savings signal is thereby split across three rows of the table rather than concentrated in one, and a reader taking the ranking at face value would underestimate how dominant that single quantity is.

A Random Forest is not harmed in its predictive accuracy by collinear inputs the way a linear model is, so this redundancy costs little in performance. It costs a great deal in interpretability, which is the stated purpose of computing SHAP values at all. Removing `expense_ratio` — or equivalently, removing `savings_ratio` — would leave the model's accuracy essentially unchanged while making the explanation honest. This was identified after the model was trained and the results generated; it is recorded here and in Section 26 rather than silently corrected, because correcting it would require regenerating every figure in Section 23.

`financial_discipline_score` compounds the issue, since it takes `savings_rate` as its highest-weighted component at 0.40. The second-ranked feature is thus partly a function of the first.

14.5 Leakage

Two columns were excluded from the feature set on leakage grounds. `financial_decision_score` is a weighted sum of savings rate, credit score, debt burden, investment experience and emergency-fund coverage — the same five quantities that determine `risk_score`, from which the target label is cut. Supplying it as a feature would hand the model a noisy copy of the label. `expected_savings_increase` is likewise a function of the generator's internal discipline variable.

The five one-hot columns of `investment_preference` are also dropped, since that column serves as the secondary target.

Verification confirms that `feature_columns.json` contains none of these seven columns. The 48 features decompose as:

Origin	Count
Raw numeric columns passed through unchanged	16
Ordinal encodings	5
One-hot dummies (gender, occupation, marital status, location, employment type)	20
Derived features	7
Total	48

The five original categorical columns are absent, superseded by their encoded forms. Of the 25 dummies produced by one-hot encoding six nominal columns, the 5 belonging to `investment_preference` are excluded, leaving 20.

14.6 What Was Not Engineered

No interaction terms were constructed, since a tree ensemble discovers interactions through successive splits and explicit products would be redundant. No temporal features exist, because the dataset has no time dimension: each record is a single snapshot with no history. No text or free-form fields are present, so no embedding or vectorisation step is required.

Feature scaling was fitted and persisted as `scaler.pkl` during preprocessing but is not applied to the models actually deployed. Tree ensembles are invariant to monotone transformations of individual features, so standardisation changes nothing for the Random Forest that was selected. It would have mattered for the neural network, which was evaluated during model selection and rejected.

Figure 14.1 — Scatter of `savings_ratio` against `expense_ratio`, showing the exact anti-diagonal. A single plot demonstrates the redundancy argued in Section 14.4 more convincingly than the correlation coefficient. Generate with a two-line matplotlib block over `processed_dataset.csv`. Place in Section 14.4.

Figure 14.2 — Global SHAP summary (beeswarm). Already generated at `ml/plots/shap_summary.png`. Place in Section 14.4 alongside the importance table.

Figure 14.3 — Distribution of the two composite scores, `behavioral_composite_score` and `financial_discipline_score`, as overlaid histograms. Note in the caption that the behavioural index is left-skewed with mean 30.7, reflecting that most synthetic records have limited investment experience. Existing boxplots at `data/processed/plots/financial_discipline_vs_risk.png` may serve instead. Place in Section 14.3.

Table 14.1 — Feature provenance. Forty-eight rows grouped by origin: raw numeric (11), ordinal encoded (5), one-hot (25), derived (7). Mark each as used or excluded, and give the exclusion reason for the six leakage columns. Place as an appendix, referenced from Section 14.5.

15. Machine Learning Model Selection

15.1 Framing the Problem

Predicting `risk_category` is a three-class classification over 48 tabular features, with classes distributed 35 / 45 / 20. The data is entirely tabular: no images, no sequences, no text. There are 22,000 records, which is enough to fit a moderately complex model and not enough to justify one that must learn its own representations.

The class imbalance is mild but material. A classifier that always answered *Medium* would score 45% accuracy, so accuracy alone cannot demonstrate that a model has learned anything. Weighted F1 was adopted as the selection criterion, and both are reported.

15.2 Candidates

Five classifiers were trained and compared, chosen to span the approaches that are actually competitive on tabular data rather than to enumerate algorithms for their own sake.

Random Forest — bagged decision trees with feature subsampling at each split. Robust to feature scale, tolerant of collinearity, and cheap to explain with `TreeExplainer`.

XGBoost, **LightGBM** and **CatBoost** — three gradient-boosting implementations. Boosting fits trees sequentially against the residual of the ensemble so far, which typically extracts more signal than bagging when signal exists to be extracted.

Neural network — a two-hidden-layer perceptron, 128 and 64 units, with early stopping. Included as a control rather than a serious contender: on tabular data of this size, gradient-boosted trees generally outperform dense networks, and confirming that on this dataset is more useful than assuming it.

A logistic regression baseline was not fitted, which is an omission. It would have established whether the non-linear models were earning their complexity, and its absence is noted in Section 26.

15.3 Protocol

The processed table was split 80/20 with stratification on the label under `random_state=42`, giving 17,600 training and 4,400 test records. Every model saw the same partition.

All five were first fitted with library-default hyperparameters and scored on the held-out set. Selection was made on weighted F1 against those defaults, so that the comparison was between algorithms rather than between tuning budgets. The Random Forest won at $F1 = 0.5949$, ahead of LightGBM at 0.5830 and CatBoost at 0.5827.

The winner alone was then tuned by randomised search over 20 candidate configurations with three-fold cross-validation, which raised its weighted F1 to 0.6006. The selected configuration:

Hyperparameter	Value
n_estimators	200
max_depth	10
min_samples_split	5
min_samples_leaf	1
max_features	sqrt
criterion	gini
bootstrap	True

Five-fold stratified cross-validation on the tuned model gives weighted F1 = **0.5996 ± 0.0032**, confirming that the held-out figure is not an artefact of one fortunate partition.

Feature scaling was fitted during preprocessing and saved, but is not applied. Tree ensembles are invariant to monotone rescaling of individual features, so standardisation would change nothing for the selected model. It would have mattered for the neural network, and its absence there is a confound: the network was evaluated on unscaled inputs, which disadvantages it. Since the network was included as a control and lost by a wide margin, this does not affect the selection, but the reported figure understates what a properly scaled network would achieve.

15.4 Comparison

Model	Accuracy	Precision (w)	Recall (w)	F1 (w)	ROC AUC (ovr, w)
Random Forest (tuned)	0.6039	0.6177	0.6039	0.6006	0.7461
LightGBM	0.5845	0.5923	0.5845	0.5830	0.7359
CatBoost	0.5841	0.5906	0.5841	0.5827	0.7290
XGBoost	0.5620	0.5686	0.5620	0.5609	0.7164
Neural Network	0.5332	0.5447	0.5332	0.5000	0.6550

One caveat attaches to this table, and it should not be buried. The Random Forest row reports the **tuned** model, while the other four report **untuned defaults**, because `train.py` overwrites the winner's entry with its post-tuning metrics. The table therefore compares a tuned model against four that were not tuned, and overstates the Random Forest's margin by roughly half a point of F1. The selection decision is unaffected — it was taken before tuning, on the untuned figures, where the Random Forest also led — but a reader comparing rows is not comparing like with like. The untuned Random Forest scored F1 = 0.5949.

15.5 Why the Random Forest Won

Gradient boosting outperforming bagging is the ordinary expectation on tabular problems, and here it did not. The explanation lies in the structure of the label rather than in any property of the implementations.

As Section 12.4 sets out, `risk_category` is a threshold on a linear score to which Gaussian noise of standard deviation 8.0 has been added, against a signal whose standard deviation is 7.64. Roughly half the variance in the underlying score is irreducible. Boosting's advantage is its capacity to fit residual structure across successive rounds; when the residual is predominantly noise, that capacity becomes a liability, and the boosted models overfit it. Bagging averages independently grown trees and cannot chase a residual it never computes.

Section 23 makes this concrete by deriving the Bayes-optimal accuracy for this label — the ceiling that no classifier can exceed — and showing that the Random Forest reaches 98.9% of it. There was very little residual structure left for boosting to find.

15.6 The Secondary Model

A second Random Forest was trained to predict `investment_preference` across five classes, using the same features and an identical protocol but no hyperparameter search.

It achieves accuracy 0.2586, weighted F1 0.1892 and ROC AUC 0.4949. The majority-class baseline is 0.2869 and chance-level ROC AUC is 0.5000. **The model performs worse than always answering "Mutual Funds", and its ranking of classes is indistinguishable from random.**

This is not a tuning failure and could not be repaired by a better algorithm. Section 22.3 traces it to the construction of the label.

Figure 15.1 — *Model comparison bar chart*, five models on the x-axis, grouped bars for accuracy / weighted F1 / ROC AUC. Annotate the Random Forest bar to indicate it is the tuned model, per the caveat in Section 15.4. Place in Section 15.4.

Figure 15.2 — *ROC curves, one-vs-rest*, three curves for Low / Medium / High from the tuned Random Forest, with the macro-average and the diagonal. Generate from `predict_proba` on the test split. Place in Section 15.4.

Table 15.1 — *Randomised search space*, listing the distribution sampled for each hyperparameter alongside the selected value. Source from `train.py`. Place in Section 15.3.

16. Implementation Details

16.1 Repository Organisation

The project is a single repository containing five deployable or executable concerns, separated by directory rather than by package boundary.

```
behavioral-risk-assessment/  
  data/  
    raw/          generate_dataset.py, dataset.csv, dataset_description.md  
    processed/    preprocess.py, processed_dataset.csv, scaler.pkl, plots/  
  ml/  
    train.py, inference.py, test_inference.py  
    models/       risk_model.pkl, investment_model.pkl, label encoders,  
                  feature_columns.json, model_metadata.json,  
                  preprocessing_params.json  
    plots/        shap_summary.png, feature_importance.png  
  backend/  
    app/          main.py, config.py, database.py,  
                  routers/, services/, schemas/, models/, middleware/  
    Dockerfile, requirements.txt, .env.example  
  frontend/  
    src/          api/, pages/, components/, contexts/, types/  
    Dockerfile, nginx.conf, package.json  
  docs/  
    api-contract.json, report/  
  tests/  
    test_api.py, test_inference.py  
  docker-compose.yml
```

The `data` and `ml` directories are executed offline and produce artefacts. The `backend` and `frontend` directories are long-running services. Nothing in `backend` writes to `ml`; the dependency runs one way only.

16.2 Toolchain

Layer	Technology	Version
Frontend framework	React	19
Language	TypeScript	6.0
Build tool	Vite	8
Styling	TailwindCSS	3.4
Charts	Recharts	3.9
Forms and validation	React Hook Form 7, Zod 4	
Routing	React Router	7
Backend framework	FastAPI	≥ 0.110
ORM	SQLAlchemy	2.0
Validation	Pydantic	2
Database	PostgreSQL 15 / SQLite 3	
ML	scikit-learn, XGBoost, LightGBM, CatBoost, SHAP	
Orchestration	Docker Compose	

Radix UI primitives underpin the select, dialog, progress and tooltip components. Linting uses `oxlint`.

16.3 Configuration

Runtime configuration is read from environment variables through a `pydantic-settings` model, with `.env` as a fallback source and defaults suitable for local development.

Variable	Default	Purpose
<code>DATABASE_URL</code>	<code>sqlite:///./risk_assessment.db</code>	connection string
<code>ML_PATH</code>	<code>/ml</code>	location of the inference module
<code>SECRET_KEY</code>	<code>dev-secret-key</code>	declared, currently unused
<code>LOG_LEVEL</code>	<code>INFO</code>	logging verbosity
<code>VERSION</code>	<code>1.0.0</code>	reported by <code>/health</code>

`SECRET_KEY` is defined in the settings model but referenced nowhere in the codebase. It was provisioned for a session or token mechanism that was not implemented, and it is retained only because removing it would require a corresponding change to the deployment templates. It signs nothing.

16.4 Deployment

Three containers are composed. PostgreSQL exposes 5432 and declares a `pg_isready` health check; the backend declares `depends_on` with `condition: service_healthy`, so it does not start against a database still initialising. The backend mounts `./ml` at `/ml` — this bind mount is what makes the model available to the service, and the container will start without it but report `model_loaded: false` from `/health` and return 503 from `/predict`.

The frontend is built in a multi-stage image: a Node stage compiles the TypeScript and produces static assets; an nginx stage serves them. The nginx configuration proxies `/api/` to the backend service and rewrites unmatched paths to `index.html`, which is required for client-side routing to survive a page reload.

Two adjustments were necessary to make the compose stack build. The frontend Dockerfile originally invoked `npm ci`, which aborts when `package-lock.json` and `package.json` disagree; the lockfile in the repository had drifted, and the instruction was changed to `npm install`. Separately, the backend image did not install NumPy, pandas, scikit-learn or SHAP, because `requirements.txt` listed only the web dependencies. The ML packages were added, since the backend imports the inference module directly into its own process.

16.5 Offline Pipeline Execution

The three offline scripts are run in sequence and are individually idempotent under their fixed seed.

```
cd data/raw      && python3 generate_dataset.py    # → dataset.csv
cd ../processed  && python3 preprocess.py          # → processed_dataset.csv
                                                         # preprocessing_params.json
cd ../../ml      && python3 train.py                # → models/*.pkl, metadata
```

Re-running `preprocess.py` after the parameter-persistence change was verified to leave `processed_dataset.csv` byte-identical, which is the reason the existing trained model remained valid and no retraining was required when the serving defects described in Section 22 were repaired.

16.6 Testing

Eleven tests exist across two files.

`tests/test_inference.py` holds five unit tests exercising `predict()` directly, without a server. They assert that all twelve expected keys are present in the returned dictionary, that `risk_category` is one of the three permitted labels, that both confidence values lie in `[0, 1]`, that the three scores lie in `[0, 100]`, and that at least one recommendation is produced.

`tests/test_api.py` holds six integration tests requiring a running service. They cover `/health`, `/model/info`, a successful `/predict`, an assessment round trip through `POST /assessment` followed by `GET /assessment/{id}`, the `/analytics` aggregate, and one negative case asserting that an `age` of 10 is rejected with HTTP 422.

Coverage is thin and should be described as such. There are no tests for the preprocessing scripts, none for the frontend, and none that would have caught any of the five feature-engineering defects discussed in Section 22 — because every one of those defects lay in the agreement between two code paths, and no test compared them. The equivalence check that eventually exposed them, described in Section 9.7, is not part of the automated suite. Promoting it to a regression test would be the single most valuable addition to this repository.

16.7 Defects Identified During Documentation

Writing this report required reading the implementation closely, and doing so surfaced defects that the test suite had not. Three are recorded here because they affect the backend as deployed; the five feature-engineering defects are treated in Section 22, and the data-loading defect in Section 12.5.

The `/model/info` endpoint reads an absolute path from the author's machine. `backend/app/routers/health.py` defines `MODEL_METADATA_PATH` as `/Users/chiragmali/Documents/.../ml/models/model_metadata.json`. Inside the container this path does not exist, so the endpoint returns `{"error": "model_metadata.json not found"}` with HTTP 200 rather than the metadata. It works when the service runs directly on the development machine and fails under Docker Compose, which is the documented deployment path. It should resolve the path relative to `ML_PATH`, as `ml_service` does.

The local-development fallback in `ml_service` resolves one directory too high. When `/ml` is absent the module constructs a fallback path by ascending four levels from `app/services/`, which lands on the repository's parent directory rather than the repository root. The resulting path does not exist, the import fails, and `/predict` returns 503. The defect is invisible under Docker Compose, where `/ml` is always mounted and the fallback never executes.

The audit middleware performs a synchronous database write inside an asynchronous handler. `LoggingMiddleware.dispatch` is a coroutine, but it opens a SQLAlchemy session and commits without yielding, which blocks the event loop for the duration of the write on every request. The inline comment describes the write as "non-blocking best-effort"; it is best-effort, in that a failure is swallowed, but it is not non-blocking. The middleware additionally persists the entire request body to `audit_logs`, which for `POST /assessment` means storing twenty-six fields of a user's financial position in a table that carries no access control.

None of the three affects the accuracy figures reported in Section 23, because none touches the prediction path in the configuration under which those figures were produced.

Figure 16.1 — *Repository tree*, rendered as a directory listing. Trim to two levels. Place in Section 16.1.

Screenshot 16.1 — *docker compose up reaching a healthy state*, showing all three containers running and the backend log line `ML models loaded and ready`. Place in Section 16.4.

Screenshot 16.2 — *Terminal output of `python3 -m pytest tests/ -v`*, showing all eleven tests passing. Place in Section 16.6.

Table 16.1 — *Environment variables*, reproducing the table in Section 16.3 with an added column stating whether each is consumed by code. Mark `SECRET_KEY` as unused.

17. Database Design

17.1 Choice of Store

The data this system produces is relational in the ordinary sense: an assessment has exactly one prediction, a prediction is meaningless without the assessment that produced it, and the analytics view is a set of aggregate queries over those two tables. A relational store was therefore the obvious fit, and PostgreSQL was selected for its native `JSONB` support, which matters because three columns hold structures whose shape is fixed but whose contents vary in length.

SQLite is retained as a development target. Both are reached through the same SQLAlchemy declarative layer, and the schema is expressed once. Where the two dialects genuinely diverge, the divergence is handled explicitly rather than papered over — the daily-count aggregation in `/analytics` inspects `db.bind.dialect.name` and emits `strftime` for SQLite or `to_char` for PostgreSQL.

17.2 Entities

Five tables are declared.

users — a session identifier, creation timestamp and originating IP address, keyed by UUID.

assessments — the twenty-six submitted inputs stored as first-class columns, together with a creation timestamp and a `status` field taking `pending`, `completed` or `failed`. A nullable foreign key points at `users`.

predictions — one row per assessment, enforced by a unique constraint on `assessment_id`. Holds the predicted risk category and its confidence, the predicted investment preference and its confidence, the three derived scores, and four `JSON` columns: `risk_probabilities`, `shap_values`, `feature_importance`, `recommendations` and `behavioral_biases`.

behavioral_scores — a narrow table storing a `score_type` and `score_value` per assessment. Three rows are written per prediction, one each for the behavioural composite, financial discipline and decision scores.

audit_logs — endpoint, HTTP method, request body, response status and duration in milliseconds, written by middleware on every request.

17.3 Entity–Relationship Diagram



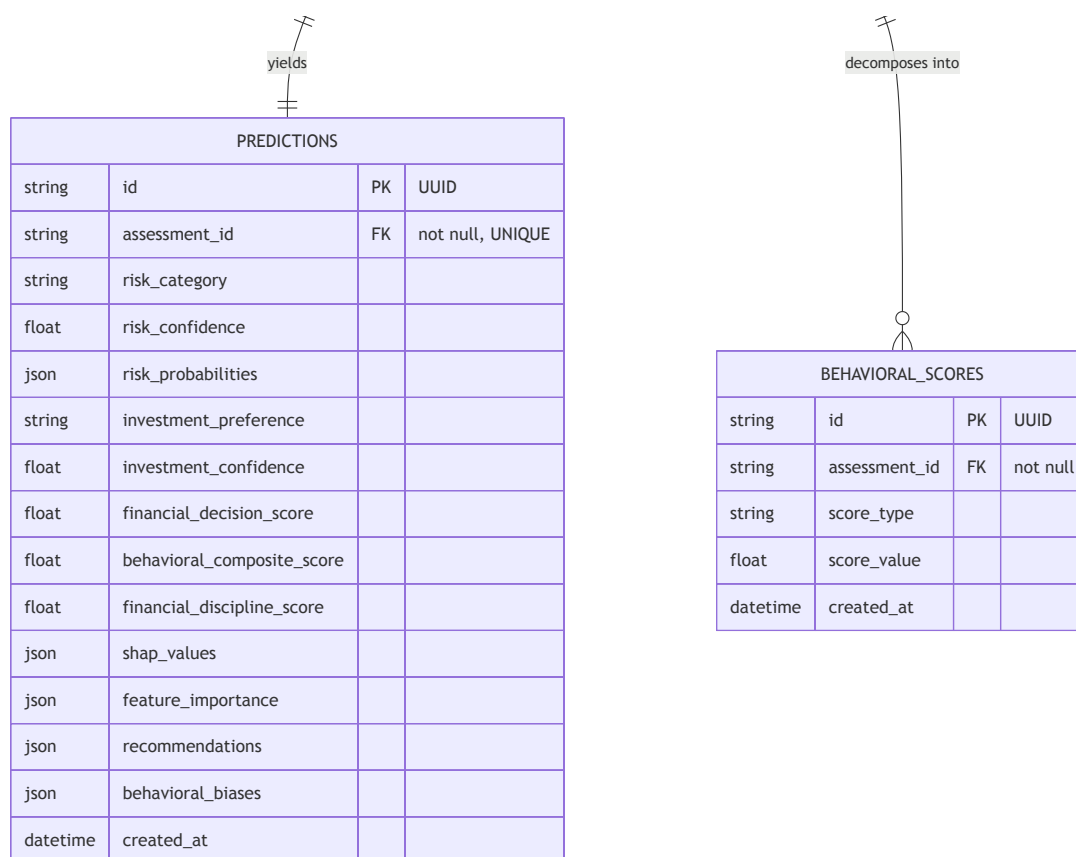


Fig. 17.1 — Entity-relationship diagram.

`AUDIT_LOGS` is drawn without a relationship because it holds none. It records HTTP traffic rather than domain objects, and deliberately carries no foreign key to `assessments`; an audit trail that could be broken by a cascade delete would defeat its purpose.

17.4 Normalisation

The schema is in first normal form: every column holds a single value, and the repeating structures are confined to the `JSON` columns, which store an atomic document rather than a repeating group of relational attributes.

It satisfies second normal form trivially, since every primary key is a single `UUID` column and no partial dependency on a composite key is possible.

Third normal form holds for `predictions`, `users` and `audit_logs`, where every non-key attribute depends on the key alone. **It does not strictly hold for `assessments`**. Two functional dependencies exist among the non-key attributes. `savings_rate` is determinable from `monthly_income` and `monthly_expenses`, and `income_level` is a coarse banding of `monthly_income`. A purist would remove `savings_rate` and derive it in a view.

Both were kept, and the reason is worth stating rather than hiding. `savings_rate` is what the user asserted, and it need not equal the value implied by the income and expenses they also reported — people round, and they exclude irregular income. Storing the assertion preserves what was actually submitted, which is the property an audit trail requires. The same argument applies to `income_level`, which is self-reported rather than computed. The redundancy is accepted as the cost of recording the input faithfully.

A second and less defensible redundancy exists between `behavioral_scores` and `predictions`. The three score values written to `behavioral_scores` already appear as columns on the corresponding `predictions` row. The narrow table exists to support querying scores by type without knowing the column names, but it stores derived data that is recoverable from elsewhere, and it must be kept consistent by application code rather than by a constraint. Were the schema to be revised, `behavioral_scores` would be replaced by a view.

17.5 Keys, Constraints and Integrity

Primary keys are UUIDs generated in Python at insert time rather than sequences generated by the database. The motivation is that an identifier can be minted before the row exists, which simplifies the write path, and identifiers do not leak the number of assessments the system has processed.

The one-to-one relationship between `assessments` and `predictions` is enforced by a unique constraint on `predictions.assessment_id`, not merely by the ORM's `uselist=False`. The database rejects a second prediction for an assessment regardless of what the application attempts.

Referential integrity is declared through foreign keys on `assessments.user_id`, `predictions.assessment_id` and `behavioral_scores.assessment_id`. No cascade behaviour is specified, so a delete against a referenced parent raises rather than propagating. Since the application never deletes, this has not been exercised.

Write consistency for the assessment path relies on the SQLAlchemy session's transaction. The prediction row and the three score rows are added and committed together, so a failure part-way leaves none of them. The assessment row is committed earlier, which is what makes the `failed` status meaningful — the row survives to record that a prediction was attempted and did not complete.

17.6 Known Deficiencies

Three shortcomings in the current schema should be recorded rather than glossed over, because each would need addressing before the system carried real traffic.

The `users` table is never written to. It is declared, related to `assessments`, and created at startup, but no code path instantiates a `User`, and `assessments.user_id` is consequently always null. Authentication was scoped out of the implementation, and without it there is no principal to record. The table is therefore vestigial, and the relationship it participates in is unexercised. It is retained because the analytics and history features would attach to it directly once sessions exist.

No secondary indexes are declared. Every index in the schema is the one implied by a primary or unique key. The queries that will degrade first are the history listing, which orders `assessments` by `created_at` descending, and the analytics aggregations, which group `predictions` by `risk_category` and `investment_preference`. At the current data volume the sequential scans are imperceptible. An index on `assessments.created_at` and on the two grouped columns of `predictions` would be the first change under load.

Schema migrations are not in use. An `alembic/` directory exists but is empty, and the schema is materialised at startup by `Base.metadata.create_all()`. That call creates missing tables and silently ignores tables whose definition has drifted, so an altered column type will never be applied to an existing

database. For development against a disposable SQLite file this is adequate. For a PostgreSQL instance holding data it is not, and generating an initial Alembic revision is the correct next step.

Figure 17.1 — *Entity–relationship diagram*. Rendered from the `erDiagram` source in §17.3. If the `assessments` entity makes the figure too wide for the printed page, collapse its twenty-six input columns into three grouped rows labelled *Demographic (10)*, *Financial (10)* and *Lifestyle (6)*, and give the full column list in Table 17.1.

Table 17.1 — *Column dictionary for `assessments`*. Columns: name, SQL type, source (demographic / financial / lifestyle), permitted values or range. Populate from `backend/app/schemas/assessment.py`, which carries the authoritative bounds. Place after Figure 17.1.

Table 17.2 — *Normal-form assessment per table*. Rows: the five tables. Columns: 1NF, 2NF, 3NF, and a note. Mark `assessments` as violating 3NF with the reason from Section 17.4. Place at the end of Section 17.4.

18. API Description

18.1 The Contract as an Artefact

The interface between the browser and the service is defined in `docs/api-contract.json`, and that file is treated as authoritative rather than descriptive. The backend's Pydantic schemas and the frontend's TypeScript types are both required to match it, and any change to an endpoint is made to the contract first.

This discipline was adopted after a failure, not in anticipation of one. The two sides had been developed against separate mental models of the interface, and six divergences accumulated before anything was tested end to end: the frontend expected a flat prediction object where the backend returned an envelope; it read a field named `confidence` where the backend emitted `risk_confidence`; it posted assessments to `/assessments` where the route was `/assessment`; and three of the analytics field names disagreed. Each was individually trivial and collectively fatal — the application compiled, the backend passed its own tests, and no request succeeded. Consolidating the interface into one document that both sides are checked against removed the class of error rather than the instances of it.

18.2 Endpoints

Seven routes are served from `http://localhost:8000`.

Method	Path	Persists	Response
GET	<code>/health</code>	no	bare object
GET	<code>/model/info</code>	no	bare object
POST	<code>/predict</code>	no	enveloped
POST	<code>/assessment</code>	yes	enveloped, with <code>assessment_id</code>
GET	<code>/assessment/{id}</code>	no	enveloped, with <code>assessment_id</code>
GET	<code>/assessments</code>	no	enveloped list, with <code>total</code>
GET	<code>/analytics</code>	no	bare object

Interactive documentation is generated automatically by FastAPI from the Pydantic schemas and served at `/docs`. Because the schemas are the same objects that perform validation, the published documentation cannot drift from the enforced behaviour.

18.3 Response Envelope

The four prediction and assessment endpoints return:

```
{ "success": true, "data": { ... } }
```

POST `/assessment` and GET `/assessment/{id}` add `assessment_id` as a sibling of `data`, not inside it. GET `/assessments` substitutes `total` and `items` for `data`.

The remaining three endpoints return their payload unwrapped. This is an inconsistency rather than a design decision, and it should be recorded as such: a client cannot determine from the shape of a response whether it needs to unwrap. The frontend absorbs the difference inside `src/api/assessments.ts`, where `postPredict` returns `response.data.data` while `getAnalytics` returns `response.data`. No page component sees the distinction, so the cost is contained, but a second consumer of this API would have to read the contract to discover which convention applies where.

18.4 Request Schema

All four write endpoints accept the same body, twenty-six fields validated by `AssessmentInput`. Categorical fields are `Literal` types constrained to the exact strings present in the training vocabulary; numeric fields carry explicit bounds. The model declares `extra = "forbid"`, so an unrecognised key is an error rather than an ignored field.

An example request:

```
curl -X POST http://localhost:8000/predict \
-H 'Content-Type: application/json' \
-d '{
  "age": 30, "gender": "Male", "education": "Graduate",
  "occupation": "Salaried", "income_level": "Middle",
  "marital_status": "Single", "dependents": 0,
  "location": "Urban", "employment_type": "Full-Time",
  "years_of_experience": 5,
  "monthly_income": 50000, "monthly_expenses": 35000,
  "savings_rate": 30, "emergency_fund_months": 3,
  "total_debt": 100000, "loan_amount": 500000,
  "credit_score": 720, "investment_experience_years": 3,
  "investment_frequency": "Monthly", "insurance_coverage": "Basic",
  "shopping_frequency": "Monthly", "online_spending_pct": 30,
  "luxury_spending_pct": 10, "subscription_count": 3,
  "gaming_expenses_monthly": 500, "travel_expenses_annual": 20000
}'
```

Note that `savings_rate` is supplied as a percentage. The inference module converts it to the fraction the model was trained on, as Section 13.5 describes.

18.5 Response Schema

The `data` object returned by `/predict`, `/assessment` and `/assessment/{id}` has twelve keys:

Key	Type	Meaning
risk_category	string	Low, Medium or High
risk_confidence	float	probability of the predicted class, 0–1
risk_probabilities	object	probability across all three classes
investment_preference	string	FD, Mutual Funds, Stocks, Gold or Crypto
investment_confidence	float	0–1
financial_decision_score	float	0–100
behavioral_composite_score	float	0–100
financial_discipline_score	float	0–100
shap_values	object	ten features → attribution magnitude (always ≥ 0; see §21.4)
feature_importance	object	ten features → global importance
recommendations	array	3–5 strings, rule-generated
behavioral_biases	array	rule-detected, may be empty

`investment_preference` and `investment_confidence` are returned by the API and rendered by the interface, but the model producing them does not work. Section 22 shows that its accuracy of 0.259 falls below the 0.287 majority-class baseline, and that no model could do materially better given how the label was constructed. The fields remain in the contract; the report does not claim they are meaningful.

18.6 Error Handling

Status	Condition
422	request body fails schema validation; response names the offending field
404	GET <code>/assessment/{id}</code> for an unknown identifier
500	prediction raised during <code>POST /assessment</code> ; the assessment row is marked <code>failed</code>
503	<code>POST /predict</code> when the inference module failed to import at startup

Validation errors are the common case and are informative. Submitting `"education": "Bachelor's"` — a value the interface once offered but the model has never seen — produces a 422 naming `education` and listing the four permitted literals. This is the correct behaviour: a silently accepted synonym would be encoded as the training median and would produce a plausible-looking prediction from an input the model cannot represent.

GET `/model/info` is an exception to the table. When it cannot locate the metadata file it returns `{"error": "..."} with HTTP 200 rather than a 5xx. Because the path it consults is hardcoded to an absolute location on the development machine, this is the response it produces under Docker Compose. Section 16.7 records the defect.`

18.7 Analytics

`GET /analytics` aggregates over stored assessments and returns totals, the distribution of predicted risk categories and investment preferences, the mean of each of the three scores, and a per-day assessment count.

The per-day count is the one query that could not be written portably. SQLite truncates a timestamp with `strftime('%Y-%m-%d', ...)`; PostgreSQL requires `to_char(..., 'YYYY-MM-DD')` and rejects the former outright. The service inspects `db.bind.dialect.name` and emits the appropriate expression. Before this was introduced the endpoint returned HTTP 500 under Docker Compose while passing its tests against SQLite locally — a failure mode worth noting, since the test suite exercised a database the deployment does not use.

18.8 Cross-Origin Policy and Authentication

CORS permits `http://localhost:5173` and `http://localhost:3000`, matching the Vite development server and the nginx container respectively, with credentials enabled and all methods and headers allowed.

No authentication exists. There are no credentials, no sessions and no tokens; every endpoint is unauthenticated and every stored assessment is anonymous. The `users` table described in Section 17 is never written to for exactly this reason. The consequence is that `GET /assessments` returns every assessment any visitor has ever submitted, including the twenty-six financial fields of each, to any caller. For a demonstration system with synthetic inputs this is tolerable. For a system carrying real financial disclosures it would not be, and Section 26 records it as the most serious limitation of the implementation.

Screenshot 18.1 — *Swagger UI at `/docs`, all seven endpoints collapsed and visible in one frame. Place in Section 18.2.*

Screenshot 18.2 — *An expanded `POST /predict` in Swagger, showing the generated request schema with the `Literal` enumerations. This demonstrates that the published documentation is the enforced schema. Place in Section 18.4.*

Screenshot 18.3 — *A 422 response, produced by submitting `"education": "Bachelor's"`. Place in Section 18.6.*

Figure 18.1 — *Sequence diagram for `POST /assessment`, spanning browser, router, schema validation, inference module and database. This is Figure 11.2; render it once from the Mermaid source in Section 11.2 and cross-reference.*

19. Frontend Module

19.1 Composition

The client is a single-page React 19 application written in TypeScript and bundled by Vite. Styling uses TailwindCSS with a small set of components built over Radix UI primitives — select, progress, label, separator, dialog, tooltip — which supply keyboard handling and ARIA semantics that a hand-rolled dropdown would not. Charts are drawn with Recharts. Forms are managed by React Hook Form and validated by Zod. Routing uses React Router.

Five routes are declared in `App.tsx` :

Path	View
<code>/</code>	Landing
<code>/assessment</code>	Four-step input form
<code>/results/:id</code>	Prediction breakdown
<code>/history</code>	Past assessments
<code>/analytics</code>	Aggregate dashboard

19.2 Separation from Business Logic

The frontend computes nothing. It holds no thresholds, no scoring weights, no risk bands and no model. Every number it renders arrives from the service, and its only substantive responsibility beyond presentation is input validation.

That validation duplicates the backend's, deliberately. The Zod schema in `Assessment.tsx` enumerates the same literal values as the Pydantic `Literal` types, so a user selecting an invalid option is told immediately rather than after a round trip. The duplication is a usability affordance, not a source of truth; the backend rejects independently, and a divergence between the two is a defect in the frontend rather than a relaxation of the rule.

The two did diverge. An earlier build of the form presented `occupation` and `location` as free-text inputs, and offered dropdown options — `Bachelor's`, `Master's`, `Upper-Middle`, `Full-time`, `Daily` for investment frequency, `Health Only` for insurance — that appear nowhere in the training vocabulary. Every submission carrying one of these was rejected with HTTP 422. The fields are now `Select` components restricted to the ten permitted literal sets given in Section 13.3, and the Zod schema uses `z.enum` rather than `z.string`, so an invalid value cannot survive the type checker.

19.3 Type Safety Against the Contract

All request and response shapes are declared in `src/types/index.ts`, and every call passes through `src/api/assessments.ts`. No page component constructs a URL or touches Axios. This confines the effect of an interface change to two files, and `npm run build` runs `tsc -b` before Vite, so a type that no longer

matches its usage fails the build rather than the runtime.

The API layer also absorbs the envelope inconsistency described in Section 18.3. `postPredict` returns `response.data.data`; `getAnalytics` returns `response.data`. Components receive the inner payload in both cases and are unaware that the two endpoints disagree about shape.

19.4 Views

Landing presents the system and routes to the assessment.

Assessment is a four-step wizard over the twenty-six inputs, grouped as demographics, financial profile, lifestyle and review. Each step is validated with `trigger()` on its own field subset before the user may advance, so errors surface on the step that produced them rather than at submission.

Results renders the prediction. A Recharts `RadialBarChart` displays the confidence of the predicted class, coloured green, amber or red by risk band. A horizontal bar chart shows the ten SHAP attributions. The component colours each bar blue for a positive value and red for a negative one, and captions the chart accordingly, but no bar is ever red: the service returns absolute magnitudes, as Section 21.4 explains. The legend describes a distinction the data does not carry. Three numeric cards show the decision, behavioural composite and discipline scores. Detected behavioural biases appear as badges, and the rule-generated recommendations as an ordered list.

History lists past assessments, each row linking to its results.

Analytics fetches `/analytics` and draws the risk and investment distributions as pie charts, the mean scores as a bar chart, and the per-day assessment count as a line chart.

Dark mode is provided by a `ThemeContext` that reads the system preference on first load and persists an override to `localStorage`.

19.5 Persistence of Submitted Assessments

The form submits to `POST /assessment`, receives an `assessment_id`, caches the payload in `sessionStorage` under that identifier for an immediate render, and navigates to `/results/{id}`. If a later visit to that route finds no cached entry — a reloaded tab, a shared link, a different browser — the page refetches through `GET /assessment/{id}`.

This was not the original behaviour, and the correction is worth recording because the failure was invisible to every test in the suite.

The form previously called `postPredict`, the endpoint that computes a prediction and deliberately does not store it. `postAssessment` existed in `src/api/assessments.ts`, was correctly typed, and was called from nowhere. The page synthesised an identifier from `Date.now()` and used it as though it were a database key.

Four consequences followed, each of which would have presented as an unrelated bug. Nothing a user entered through the interface reached the database, so the `assessments`, `predictions` and `behavioral_scores` tables held only rows created by direct `curl` calls during testing. The history view queried the database, found nothing the user had submitted, and silently fell back to a `localStorage` list

populated by an explicit "Save to History" button — so the history a user saw was a browser artefact rather than a record. The results route worked only from `sessionStorage`; a reload passed a millisecond timestamp to `GET /assessment/{id}` and received HTTP 404, which meant results were neither shareable nor durable. And the analytics dashboard aggregated a table the interface never wrote to.

The defect survived because the backend was correct. Its integration tests exercised `POST /assessment` directly and passed. The frontend compiled, and TypeScript had no opinion about which of two correctly-typed functions the page chose to call. Nothing in the system compared the endpoint the interface used against the endpoint the objectives required, because no test spanned both.

The repair changed two lines: the import, and the call site. Verification exercised the corrected path end to end against a fresh database. Submitting an assessment returned the UUID `5d0b172f-...`; `GET /assessment/{id}` resolved it; `GET /assessments` reported a total of one and listed that identifier; `GET /analytics` counted it. Inspecting the store afterwards showed one `assessments` row with `status = "completed"`, one `predictions` row carrying its SHAP attributions, and three `behavioral_scores` rows for the composite, discipline and decision scores. The `users` table remained empty, as expected in a system without authentication.

The end-to-end flow stated in the objectives of Section 7 — a user submits, the model predicts, the prediction is stored, the dashboard renders the history — is exercised by the running application.

19.6 Build and Serving

`npm run build` type-checks with `tsc -b` and then bundles with Vite. The production bundle is 922.99 kB of JavaScript, 279.68 kB after gzip, alongside 20.79 kB of CSS. The JavaScript chunk exceeds Vite's default 500 kB warning threshold, and the bulk of it is Recharts together with the D3 modules it depends on. Splitting the analytics and results routes behind `React.lazy` would bring the initial chunk under the threshold; this has not been done.

In deployment the assets are served by nginx from a multi-stage image, with `try_files $uri $uri/ /index.html` so that a reload on `/results/123` returns the application rather than a 404, and an `/api/` proxy to the backend service.

Screenshot 19.1 — *Landing page*, light and dark mode side by side, demonstrating the theme toggle described in Section 19.4.

Screenshot 19.2 — *Results page*, full view showing the radial confidence gauge, the three score cards, the SHAP bar chart with both positive and negative attributions visible, and the recommendations list. This is the single most important screenshot in the report. Place at the start of Section 19.4.

Screenshot 19.3 — *Analytics dashboard*. Submit several assessments through the form before capturing, so the distributions and the per-day line chart contain more than a single point.

Screenshot 19.4 — *Assessment form on a mobile viewport (375 px)*, demonstrating the responsive layout. Place in Section 19.4.

Figure 19.1 — *Component and data-flow diagram of the frontend*: App → route components → `api/assessments.ts` → Axios client → backend. Show `Assessment` calling `postAssessment` and `Results` falling back to `getAssessment` when the `sessionStorage` cache misses, since that pair is the subject of Section 19.5.

20. Backend Module

20.1 Framework and Rationale

The service is written with FastAPI. Two properties motivated the choice over the alternatives considered.

The first is that validation and documentation derive from the same declaration. A Pydantic model annotated with `Literal["Low", "Middle", "High"]` both rejects a non-conforming request at runtime and appears in the OpenAPI schema as an enumeration. There is no second artefact to keep synchronised, and the interactive documentation at `/docs` cannot describe behaviour the service does not enforce.

The second is that the machine learning module is Python. A Java or Node service would have had to reach the model across a process boundary — through a Python sidecar, or by exporting the Random Forest to ONNX or PMML — and each of those introduces a serialisation format that must agree with the training pipeline about feature ordering and dtype. Given that the preceding sections document five defects arising from exactly this class of disagreement between two representations of the same computation, adding a third representation would have been unwise. FastAPI imports `inference.predict` and calls it as a function.

20.2 Layering

Four layers, enforced by convention rather than by tooling.

Routers (`app/routers/`) parse the request and shape the response. They contain no SQL and no arithmetic. Each router is registered in `main.py` with a tag, which is what groups the endpoints in the generated documentation.

Schemas (`app/schemas/assessment.py`) declare the request and response contracts. All validation lives here and nowhere else; a router never inspects a field value.

Services (`app/services/`) hold the logic. `assessment_service` is the only module that issues database queries, exposing named operations such as `create_assessment`, `save_prediction`, `mark_assessment_failed` and `get_analytics`. `ml_service` is a singleton that wraps the inference module and exposes `predict` and an `is_loaded` property.

Models (`app/models/db_models.py`) declare the five SQLAlchemy entities described in Section 17.

The rule that routers may not touch the session is the one that pays for itself. `POST /assessment` needs to create a row, call the model, persist a prediction and three scores, and mark the assessment complete or failed depending on the outcome. Expressing that as four named service calls keeps the router readable and keeps the transaction boundary in one place.

20.3 Model Loading

`ml_service` imports `inference.predict` once, at module import, which occurs during application startup. The import is wrapped in a broad `try / except` that records the failure rather than propagating it, so a service whose model is missing still starts, still answers `/health`, and reports `model_loaded: false`.

Requests to `/predict` then raise `RuntimeError`, which the router converts to HTTP 503.

Degrading rather than refusing to boot is deliberate. A container that exits on a missing bind mount produces a restart loop and an opaque failure; a container that starts and reports its own unhealthiness is diagnosable from `/health`.

The module locates the inference package through `ML_PATH`, defaulting to `/ml`, which is where `docker-compose.yml` mounts the `ml/` directory. A fallback path is computed when that directory is absent, and that fallback is wrong: it ascends four directory levels from `app/services/` and lands on the repository's parent rather than its root. The resulting path does not exist. The defect is masked under Docker Compose, where `/ml` is always present, and surfaces only when the service is run directly from a shell without `ML_PATH` set, in which case every prediction returns 503. Section 16.7 records it.

20.4 The Prediction Path

`POST /assessment` proceeds as follows. The body is validated against `AssessmentInput`. `assessment_service.create_assessment` writes a row with `status = "pending"` and commits, so that the attempt is durable before the model is invoked. `ml_service.predict` is called with the validated dictionary. On success, `save_prediction` writes one `predictions` row and three `behavioral_scores` rows and commits them together, and the assessment is marked `completed`. On any exception the assessment is marked `failed` and HTTP 500 is returned.

Committing the assessment row before the prediction is what makes the `failed` status meaningful. Had both been written in one transaction, a failed prediction would roll back the assessment and leave no record that the attempt occurred.

`POST /predict` performs the same validation and the same model call, and writes nothing.

20.5 Persistence

SQLAlchemy 2.0 declarative mapping backs five tables. Sessions are supplied to routers through a `get_db` dependency that yields a session and closes it in a `finally` block, so a session is released whether or not the handler raised.

The engine is constructed from `DATABASE_URL`. When that string names SQLite the engine receives `check_same_thread: False`, and a connection event handler issues `PRAGMA journal_mode=WAL` to permit concurrent readers alongside a writer. Neither adjustment applies to PostgreSQL.

Dialect divergence is handled in exactly one place. The daily-count aggregation in `get_analytics` inspects `db.bind.dialect.name` and emits `strftime` or `to_char` accordingly, because no portable spelling of date truncation exists across the two backends.

20.6 Middleware and Logging

A single middleware wraps every request. It records method, path, status and elapsed milliseconds to the application log, then attempts to write an `audit_logs` row.

Two characteristics of this middleware deserve to be stated plainly rather than glossed.

It is described in its own comment as a "non-blocking best-effort" write. It is best-effort — the write is wrapped in a bare `except` so that a logging failure cannot fail the request. It is not non-blocking. `dispatch` is a coroutine, and it opens a synchronous SQLAlchemy session and commits without awaiting, which occupies the event loop for the duration of the write. Under concurrency this serialises requests behind the audit table.

It persists the full request body. For `POST /assessment` that body is a complete financial disclosure: income, expenses, debt, credit score. These are written to a table that no endpoint exposes and no policy protects, in a system with no authentication. On synthetic inputs this is harmless. Section 26 treats it as a limitation rather than a feature.

20.7 Security Posture

There is none, and the report should not pretend otherwise.

`SECRET_KEY` is declared in the settings model and consumed by no code path. No authentication middleware exists, no route carries a dependency that inspects a credential, and the `users` table is never written. `GET /assessments` returns every stored assessment to any caller without pagination limits beyond a `limit` parameter capped at 100. CORS is restricted to two localhost origins, which is the only access control present, and it constrains browsers rather than clients.

This was a scoping decision rather than an oversight — authentication was outside the objectives listed in Section 7 — but its consequences are load-bearing for how the system may be used. The application is a demonstration operating on synthetic data. It is not deployable against real financial disclosures without an authentication layer, an authorisation model for the history and analytics endpoints, and a policy governing what the audit log retains.

Figure 20.1 — *Backend package structure*, as a layered block diagram: routers → schemas → services → models, with `ml_service` shown calling out to the mounted `ml/` directory and `assessment_service` calling the database. Place in Section 20.2.

Screenshot 20.1 — *Backend startup log*, showing `Database initialized.` followed by `ML models loaded` and `ready.` This is the evidence that the bind mount resolved and the artefacts deserialised. Place in Section 20.3.

Figure 20.2 — *State transitions of `Assessment.status`*: `pending` → `completed`, and `pending` → `failed`. A two-transition diagram. Place in Section 20.4 to support the argument about the early commit.

21. Explainable AI Module

21.1 Why Explanation Is Required Here

A risk band on its own is not actionable. Told that they are *High* risk, a user learns nothing they can act upon; told that the assessment rests chiefly on a savings rate of 4% and an emergency fund covering under three months, they learn what to change.

There is a second reason, particular to the domain. The rule-based rubrics that this system replaces, described in Section 8.5, are transparent by construction: an analyst can read the weight attached to each answer. Substituting a learned function forfeits that transparency unless something restores it. The explainability module exists to pay that debt, not to decorate the output.

21.2 Method

Attributions are computed with SHAP (SHapley Additive exPlanations), which assigns each feature the average marginal contribution it makes to the prediction across all orderings in which features could be introduced. The construction is the Shapley value from cooperative game theory, and it is the unique attribution satisfying local accuracy, missingness and consistency.

Computing it exactly is exponential in the number of features in general. For tree ensembles it is not: `TreeExplainer` exploits the tree structure to compute exact Shapley values in time polynomial in the depth and number of trees. Since the deployed model is a 200-tree Random Forest of depth 10 over 48 features, exact attribution is tractable per request, and no sampling approximation is needed.

`KernelExplainer` is retained in the code for the neural network, which is not deployed.

21.3 Global and Local Explanation

Two distinct artefacts are produced, and conflating them is a common error.

Global importance is computed once, during training, by averaging the absolute SHAP value of each feature across the test partition. It answers: on which features does this model rely, in aggregate? The result is stored in `model_metadata.json` and rendered as the beeswarm plot at `ml/plots/shap_summary.png`.

Local attribution is computed per request, for the single submitted row. It answers: for *this* person, which of their inputs drove *their* assessment? This is what the results page displays, and it is the reason SHAP is invoked at prediction time rather than merely at training time.

A user with an unusual profile can receive a local explanation that departs sharply from the global ranking, and that is the point.

21.4 The Serving Path Discards Direction

The implementation does not deliver the local explanation it promises.

`TreeExplainer.shap_values` on a multiclass model returns an array of shape `(n_samples, n_features, n_classes)` : a signed contribution from every feature toward every class. The serving code reduces it as follows:

```
sv_row = np.abs(sv[0]).mean(axis=1) # (n_features,)
```

Two operations occur here, and each destroys information.

The absolute value is taken, so a feature that pushed the prediction *away* from the assigned class becomes indistinguishable from one that pushed it *toward* that class. The mean is then taken across the three classes, so an attribution that was specific to *High* is averaged with those for *Low* and *Medium*, yielding a class-agnostic magnitude.

What the API returns is therefore not "how did this feature affect my risk assessment" but "how much did this feature matter to the model at this point, in any direction, for any class". Empirically, across 120 attributions drawn from twelve varied inputs, **not one is negative**.

The consequence propagates to the interface. `Results.tsx` colours each bar blue for a positive attribution and red for a negative one, and captions the chart "*Blue = increases risk score, Red = decreases risk score.*" No bar can ever be red. The legend describes a distinction that the data it renders does not carry, and a user reading it would reasonably conclude that every one of their top ten features increased their risk — including, in the sample below, a high credit score.

```
financial_discipline_score  +0.0455
savings_rate                +0.0162
credit_score                +0.0159
emergency_fund_months       +0.0119
savings_ratio               +0.0108
```

For this record — savings rate 30%, credit score 720, three months of emergency fund — the model predicted *Medium*. A signed, class-specific attribution would show `credit_score` and `savings_rate` pulling *away* from *High*. The magnitude presentation cannot express that, and the colour legend actively misleads.

The repair is small. Selecting the column corresponding to the predicted class and preserving its sign, rather than averaging absolute values across classes, yields the attribution the interface is already written to display:

```
k = int(model.predict(X_df)[0]) # index of predicted class
sv_row = sv[0][:, k]           # signed, class-specific
```

This was identified while writing this chapter and is not applied, because doing so would change every SHAP figure reported in Sections 14 and 22 and require regenerating them. It is recorded as a defect in Section 26, and it is the first thing that should be fixed in any continuation of this work.

The global ranking in `shap_summary.png`, computed by `train.py` over the test partition, is unaffected: averaging absolute values across a population is the correct way to summarise global importance, and that is what a beeswarm plot displays.

21.5 What the Explanations Show

Subject to Section 22.5 — that three of the top five features encode a single underlying quantity — the ranking is consistent with the generating function.

Rank	Feature	Mean abs. SHAP	Appears in <code>risk_score</code> ?
1	<code>savings_rate</code>	0.0462	yes, weight -30
2	<code>financial_discipline_score</code>	0.0407	derived from three of the five
3	<code>emergency_fund_months</code>	0.0280	yes, weight +10 (as an indicator)
4	<code>savings_ratio</code>	0.0217	proxy for <code>savings_rate</code>
5	<code>expense_ratio</code>	0.0217	complement of <code>savings_ratio</code>
6	<code>investment_experience_years</code>	0.0147	yes, weight -15
7	<code>behavioral_composite_score</code>	0.0134	partly
8	<code>credit_score</code>	0.0130	yes, weight +25
9	<code>years_of_experience</code>	0.0039	no
10	<code>age</code>	0.0033	no

Every feature that enters `risk_score` appears in the top eight, and the two that do not — `years_of_experience` and `age` — carry an order of magnitude less attribution. The model has recovered the structure it was given.

`credit_score` at rank eight is lower than its weight of +25 would suggest. The explanation lies in preprocessing: the IQR clip described in Section 9.3 truncates `credit_score` at a lower bound of 660.5, discarding the variation among poor-credit individuals precisely where that feature is most discriminative. The model cannot use information that was removed before it saw the column.

No lifestyle feature reaches the top ten. This is not a finding about human behaviour. `risk_score` is a function of five financial quantities only, so the lifestyle attributes have nothing to contribute, and Section 13.6 anticipates this. A real dataset might behave very differently, and nothing here licenses a claim either way.

21.6 Behavioural Biases and Recommendations

The interface also presents a list of detected behavioural biases and a set of written recommendations. Neither is produced by the model.

Both are generated by fixed rules over the raw inputs and the predicted class — for instance, flagging *present bias* when the savings rate falls below 10% while the subscription count exceeds five, or *loss aversion* when a Low-risk assessment coincides with substantial investment experience. Nine bias rules and eleven recommendation branches are implemented, of which three to five typically fire for a given record.

These are heuristics drawn from the behavioural-finance vocabulary of Section 3, not inferences. They are deterministic given the input, they were not validated against any labelled data, and no claim is made that a user so flagged exhibits the bias named. They are presented in the report as an illustrative layer, and they

are the component of this system with the weakest evidentiary basis.

Figure 21.1 — *Global SHAP beeswarm*. Already generated at `ml/plots/shap_summary.png` . Place in Section 21.5.

Figure 21.2 — *Top-20 features by mean absolute SHAP*. Already generated at `ml/plots/feature_importance.png` . Place alongside Figure 21.1.

Screenshot 21.1 — *Local SHAP bar chart on the results page*, with the "Blue = increases risk" legend visible. Caption it to state that no bar is red, and cross-reference Section 21.4. Used honestly, this screenshot documents the defect rather than the feature.

Figure 21.3 — *Intended versus actual attribution for one record*. Two side-by-side bar charts: the signed, class-specific SHAP values that `TreeExplainer` produces, and the absolute cross-class means that the API returns. Generate by calling `TreeExplainer` directly on the sample in Section 21.4. This figure makes the defect legible in one glance. Place in Section 21.4.

22. Results

22.1 Headline Outcome

The tuned Random Forest classifies `risk_category` on a held-out set of 4,400 records with accuracy 0.6039, weighted F1 0.6006 and one-vs-rest ROC AUC 0.7461. Against a majority-class baseline of 0.4500, the model recovers a substantial share of the available signal.

Taken alone, sixty percent accuracy invites the reading that the model is mediocre. Section 22.2 argues that this reading is wrong, and that the correct comparison is not against 100% but against the ceiling the data itself permits.

Three further results follow: a secondary model that cannot work and the reason why, a class of defect in the serving path that no accuracy metric could have exposed, and a redundancy in the feature set that corrupts the explanations without touching the predictions.

22.2 The Model Operates at the Bayes Limit

`risk_category` is not a property of the synthetic individuals. It is a threshold applied to a continuous score which, by construction in `generate_dataset.py`, equals a weighted sum of five features plus Gaussian noise:

```
risk_score = -30·savings_rate + 25·(1 - credit_score/900)
             + 20·(min(dti,5)/5) - 15·(investment_experience/20)
             + 10·[emergency_fund < 3] + ε,   ε ~ N(0, 8)
```

Every one of those five quantities is available to the model as a feature. The deterministic component is therefore *exactly* recoverable in principle. What is not recoverable is ϵ , and ϵ has a standard deviation of 8.00 against a signal standard deviation of 7.642 — a signal-to-noise variance ratio of 0.912.

This permits the Bayes-optimal accuracy to be computed rather than guessed. For a record with signal s , the probability of each class is determined by where the two thresholds fall relative to s under the noise distribution:

```
P(Low | s) = Φ((t1 - s)/8)
P(Med | s) = Φ((t2 - s)/8) - Φ((t1 - s)/8)
P(High | s) = 1 - Φ((t2 - s)/8)
```

The thresholds were recovered numerically by solving for the values at which the induced class proportions equal the observed 35 / 45 / 20, giving $t_1 = -11.074$ and $t_2 = 2.598$. A Bayes-optimal classifier assigns the argmax of these three probabilities. Averaging its success probability across all 22,000 records yields the ceiling.

Quantity	Accuracy
Majority-class baseline	0.4500
Tuned Random Forest, held-out	0.6039
Bayes-optimal ceiling	0.6104

The model attains **98.9%** of the maximum accuracy achievable by any classifier on this label. The remaining 0.65 points are not a modelling deficiency; they are the price of a label that is roughly half noise.

Two independent observations corroborate that the Random Forest has converged on the Bayes decision rule rather than merely landing near its accuracy.

The predicted class distribution matches. The Bayes rule, faced with a noisy threshold, systematically retreats toward the central class: near either boundary the middle band is the safest guess, so *High* is predicted less often than it occurs. The model reproduces this almost exactly.

Class	True share	Bayes rule predicts	Random Forest predicts
Low	35.0%	32.9%	31.4%
Medium	45.0%	54.4%	56.2%
High	20.0%	12.7%	12.5%

The under-prediction of *High* is not a bias the model acquired from imbalanced data. It is the optimal response to threshold noise, and the Bayes rule commits it too.

Errors respect the ordinal structure. Of 4,400 test records, 2,657 are classified correctly and 1,706 of the 1,743 errors fall between adjacent bands. Only **37 records — 0.84% —** are misclassified across two bands, confusing *Low* with *High*.

	pred Low	pred Medium	pred High
true Low	906	617	17
true Medium	454	1373	153
true High	20	482	378

A model that had failed to learn the underlying score would scatter its errors. This one places them where the noise places them: at the thresholds.

The cost is borne by recall on the minority class. *High* is identified with precision 0.6898 but recall 0.4295 — the model is right when it says *High*, and says it too rarely. For a risk-screening application that asymmetry runs the wrong way, and Section 24 takes it up.

22.3 The Investment-Preference Model Cannot Work

The secondary classifier reports accuracy 0.2586, weighted F1 0.1892 and ROC AUC 0.4949 across five classes. It is beaten by the majority-class baseline of 0.2869, and its ROC AUC is indistinguishable from the 0.5000 of a coin.

The cause is in `generate_dataset.py`, at the point where the label is created:

```
def inv_pref(risk):
    if risk == 'Low':    return choice(opts, p=[0.40,0.25,0.10,0.20,0.05])
    elif risk == 'Medium': return choice(opts, p=[0.15,0.35,0.25,0.15,0.10])
    else:                return choice(opts, p=[0.05,0.20,0.35,0.10,0.30])
```

The draw is conditioned on `risk_category` and on nothing else. Age, income, investment experience, luxury spending — none of the twenty-six inputs enters. The label is a random sample from one of three fixed distributions, and it therefore carries no information about the features beyond what `risk_category` already encodes.

The ceiling follows directly. A classifier that knew each record's true risk band with certainty could do no better than always guess that band's modal preference:

```
ceiling | true risk =  $\sum_k P(\text{risk}=k) \cdot \max(p_k) = 0.3675$ 
```

And no classifier knows the true band; the best available estimate is 60.4% accurate. Propagating that gives an achievable ceiling of approximately **0.3210**.

Quantity	Accuracy
Random Forest achieved	0.2586
Majority-class baseline	0.2869
Ceiling given <i>predicted</i> risk	≈ 0.3210
Ceiling given <i>perfect</i> risk	0.3675

Even a perfect classifier would be wrong on nearly two records in three. The model's failure to beat the baseline is a separate and smaller problem — a five-class Random Forest on a label with no learnable structure will fit noise in the training partition and generalise worse than a constant — but the ceiling establishes that no amount of tuning, no alternative algorithm and no additional feature could rescue this target.

This is reported as a negative result. The system exposes `investment_preference` through its API and renders it in the interface, and this report does not claim the field is meaningful. The finding is about the construction of the label, not about behavioural finance, and it should not be read as evidence that investment preference is unpredictable in general.

22.4 Training–Serving Skew

The most consequential defects found in this project were invisible to every metric reported above, because they lay outside the code that produced those metrics.

Feature engineering was implemented twice: once in `preprocess.py`, which prepared the training data, and once in `inference.py`, which served live requests. The offline test set was evaluated through the first path. Live users were served through the second. The two disagreed in five places.

#	Feature	Training	Serving (as implemented)
1	<code>savings_rate</code>	fraction, -0.1 to 0.6	percentage, 0 to 100, passed through unscaled
2	<code>debt_to_income_ratio</code>	<code>debt / (income × 12)</code>	<code>debt / income</code> — 12× too large
3	<code>age_income_ratio</code>	<code>income / age</code>	<code>age / income</code> — inverted
4	ordinal maps	PhD , Middle , 4-level frequency	Doctorate , Medium , 5-level frequency
5	composite scores	min-max weighted, constants fitted on the training column	ad-hoc formulas invented in the serving module

The first is the most damaging, because `savings_rate` carries the largest SHAP attribution of any feature. A user reporting a 10% savings rate submitted the value `10`, which the model — trained on a column whose maximum is 0.6 — encountered sixteen standard deviations beyond anything in its experience. Decision trees do not extrapolate; every such value fell into the same terminal leaf.

The effect is visible in the model's response to that single input, holding all others fixed:

<code>savings_rate</code> submitted	Before the fix	After the fix, P(High)
0%	High (conf 0.534)	0.454
5%	—	0.438
10%	Medium (conf 0.472)	0.408
20%	—	0.345
30%	Medium (conf 0.507)	0.235
45%	—	0.152
60%	Medium (conf 0.456)	0.113
100%	Low (conf 0.466)	—

Before the repair, the predicted risk was not monotone in the savings rate and a user saving 10% of income was assessed *Medium* where the training scale placed them at *High*. After it, `P(High)` declines monotonically across the whole range, which is the behaviour the generating function requires.

The repair removed the possibility of recurrence rather than correcting the five instances. `preprocess.py` now writes every constant it fits — ordinal maps, IQR clip bounds, min-max ranges, imputation medians — to `models/preprocessing_params.json`, and `inference.py` loads that artefact instead of restating the values. No hardcoded encoding remains in the serving path.

Equivalence was then verified rather than assumed. Three hundred randomly drawn raw records were passed through the serving preprocessor and compared column by column against the rows the training preprocessor produced for the same records: all thirteen derived and encoded features agree to within 10^{-16} . Held-out records routed through the full serving path produce predictions identical to feeding the model its training-format features directly, on **every** row.

Re-running `preprocess.py` after the change left `processed_dataset.csv` byte-identical, so the trained model remained valid and no retraining was required. **The accuracy reported in Section 22.2 is now the accuracy a live user receives. Before the repair, it was not.**

The general lesson is worth stating. Every one of the five defects lived in the *agreement between two implementations of one computation*, and no test in the suite compared them. A test set drawn from the training pipeline validates the model; it says nothing about the pipeline that will serve it.

22.5 Redundancy Corrupts the Explanations

`savings_ratio` and `expense_ratio` are exact complements. Their definitions give `savings_ratio = 1 - expense_ratio` identically, and across the 22,000 records their Pearson correlation is -1.000000 , with `|savings_ratio + expense_ratio - 1|` never exceeding 4.4×10^{-16} . `savings_rate`, the raw column, is a third encoding of the same quantity — a noisy observation of `savings_ratio`, correlating with it at 0.944.

The global SHAP ranking splits the resulting attribution:

Rank	Feature	Mean abs. SHAP
1	savings_rate	0.0462
2	financial_discipline_score	0.0407
3	emergency_fund_months	0.0280
4	savings_ratio	0.0217
5	expense_ratio	0.0217

Ranks four and five are identical to four decimal places, which is what a tree ensemble produces when offered two perfectly anti-correlated columns: each split that might have used one is equally likely to use the other, and Shapley attribution divides between them. `financial_discipline_score` at rank two compounds the problem, since `savings_rate` is its highest-weighted component at 0.40.

A Random Forest is not harmed in accuracy by collinear inputs. The damage is confined to interpretation — and interpretation is the stated purpose of computing SHAP values at all. A reader taking the ranking at face value sees the savings signal distributed across three rows and two composites, and would substantially underestimate how completely one quantity dominates this model.

The defect was identified after the results were generated. Removing `expense_ratio` would leave accuracy essentially unchanged and make the explanation honest; doing so would require regenerating every figure in this report. It is recorded rather than silently corrected.

Figure 22.1 — *Confusion matrix heat-map*, 3×3, annotated with counts and row-normalised percentages. The off-diagonal mass sitting adjacent to the diagonal is the visual argument of Section 22.2. Place there.

Figure 22.2 — *Predicted-versus-true class distribution*, grouped bars for true / Bayes-rule / Random Forest across the three classes. This single figure carries the central claim of the chapter. Place in Section 22.2.

Figure 22.3 — *P(High) against submitted savings_rate*, two lines: before the fix (non-monotone) and after (monotone decreasing). Generate by calling `predict()` across a sweep. Place in Section 22.4.

Figure 22.4 — *Scatter of savings_ratio against expense_ratio*, showing the exact anti-diagonal. Shared with Figure 14.1; render once. Place in Section 22.5.

Table 22.1 — *Per-class precision, recall and F1* for the tuned Random Forest, with support. Values in Section 23.2. Place at the start of Section 22.2.

23. Performance Metrics

23.1 Choice of Metrics

Accuracy is reported but is not the criterion. With classes at 35 / 45 / 20, a constant predictor reaches 45%, and any accuracy figure must be read against that floor rather than against zero.

Weighted F1 is the selection criterion. It averages the per-class harmonic mean of precision and recall, weighting each class by its support, and so penalises a model that achieves accuracy by neglecting the minority band. Macro F1 is also reported, since it weights the three classes equally and therefore exposes the *High*-class weakness that the weighted figure partially conceals.

ROC AUC is computed one-vs-rest. It measures the ranking quality of the predicted probabilities independently of the decision threshold, which matters here because the interface displays a confidence value and Section 24 argues that the operating threshold should be reconsidered.

23.2 Classification Report

Tuned Random Forest, held-out partition of 4,400 records.

Class	Precision	Recall	F1	Support
Low	0.6565	0.5883	0.6205	1,540
Medium	0.5554	0.6934	0.6168	1,980
High	0.6898	0.4295	0.5294	880
Accuracy			0.6039	4,400
Macro average	0.6339	0.5704	0.5889	4,400
Weighted average	0.6177	0.6039	0.6006	4,400

The gap between macro F1 (0.5889) and weighted F1 (0.6006) is entirely attributable to the *High* class, whose F1 of 0.5294 trails the other two by nine points.

The shape of that deficit matters more than its size. *High* carries the **highest precision of any class** at 0.6898 and the **lowest recall** at 0.4295. When the model announces high risk it is usually correct; it announces it for fewer than half the individuals who warrant it. Section 22.2 shows that this is the Bayes-optimal response to threshold noise rather than a learned bias, and Section 24 argues it is nonetheless the wrong operating point for the application.

23.3 Discrimination

Metric	Value
ROC AUC, one-vs-rest, weighted	0.7461
ROC AUC, one-vs-rest, macro	0.7712

Macro AUC exceeds weighted AUC, which is the reverse of the usual pattern and is informative. AUC measures ranking, not thresholded decisions. The model ranks *High-risk* individuals well — it assigns them elevated probabilities — even though the argmax rule rarely selects that class for them. The information required to identify high-risk individuals is present in the probability vector and is discarded by taking the maximum.

This is the quantitative basis for the recommendation in Section 24 that the interface should threshold on `P(High)` rather than report the argmax.

23.4 Stability

Five-fold stratified cross-validation on the tuned model over the full 22,000 records:

Statistic	Weighted F1
Mean	0.5996
Standard deviation	0.0032

The held-out estimate of 0.6006 lies within a third of a standard deviation of the cross-validated mean, so the reported performance is not an artefact of the particular 80/20 split. A standard deviation of 0.0032 across folds also sets the scale against which differences between models should be judged: the 1.76-point F1 gap between the Random Forest and LightGBM is roughly five standard deviations and is real, whereas the 0.03-point gap between LightGBM and CatBoost is not.

That same scale governs the insurance-category defect of Section 12.5. Restoring the destroyed `None` level changes weighted F1 from 0.5955 to 0.5915, a difference of 0.0040 — approximately one standard deviation, and therefore not distinguishable from fold-to-fold variation.

23.5 Performance Against the Attainable Ceiling

Reporting accuracy against a ceiling of 1.0 is misleading for a label that is roughly half noise by construction. The Bayes-optimal accuracy derived in Section 22.2 provides the correct denominator.

Reference point	Accuracy	Interpretation
Random guessing (uniform)	0.3333	no information
Majority class (always <i>Medium</i>)	0.4500	prior only
Tuned Random Forest	0.6039	—
Bayes-optimal ceiling	0.6104	maximum attainable

Normalising the model's position between the majority baseline and the ceiling:

$$(0.6039 - 0.4500) / (0.6104 - 0.4500) = 0.9595$$

The Random Forest captures **95.9% of the accuracy that is available to be captured** above the trivial baseline, and reaches 98.9% of the absolute ceiling. Interpreting 0.6039 as a weak result mistakes the noise floor of the data for a shortcoming of the model.

This framing depends entirely on the label being synthetic and its generating function being known. It is available here precisely because the data was constructed, and it would not be available for a real dataset. That is one of the few advantages a synthetic study confers, and Section 24 discusses what is given up in exchange.

23.6 Secondary Model

Metric	Investment-preference model	Reference
Accuracy	0.2586	majority baseline 0.2869
Weighted F1	0.1892	—
ROC AUC (ovr, weighted)	0.4949	chance 0.5000
Ceiling given predicted risk	≈ 0.3210	—
Ceiling given perfect risk	0.3675	—

The model performs below the majority baseline and ranks at chance. Section 22.3 establishes that the label was constructed without reference to any feature, so this is a property of the target rather than of the estimator. No metric in this table should be cited as evidence about the predictability of investment preference in general.

23.7 Serving Latency

Latency was not measured systematically and no benchmark is reported. Each prediction loads no artefacts — models are cached at process start — but does construct a `TreeExplainer` and compute exact Shapley values for one row against a 200-tree ensemble of depth 10, which dominates the request cost. The audit-log write described in Section 20.6 executes synchronously on the event loop and adds a database round trip to every request.

Both are addressable: the explainer can be constructed once and reused, and the audit write can be dispatched to a background task. Neither was done, and the omission of any latency measurement is recorded in Section 26.

Figure 23.1 — *Per-class precision and recall*, grouped bars across Low / Medium / High. The inversion on *High* — precision high, recall low — is the point. Place in Section 23.2.

Figure 23.2 — *ROC curves, one-vs-rest*, three classes plus macro-average and the chance diagonal. Shared with Figure 15.2; render once. Place in Section 23.3.

Figure 23.3 — *Model accuracy against reference points*, a horizontal bar or number-line showing 0.3333, 0.4500, 0.6039 and 0.6104, with the interval between baseline and ceiling shaded. This visualises the 95.9% figure and is the most important chart in the chapter. Place in Section 23.5.

Figure 23.4 — *Cross-validation fold scores*, five points with the mean and ± 1 s.d. band. Place in Section 23.4.

24. Discussion

24.1 What Sixty Percent Means

The instinct on seeing 60.4% accuracy is to call the model weak. That instinct is wrong here, and correcting it is the central interpretive claim of this work.

The label was manufactured by adding noise of standard deviation 8.0 to a signal of standard deviation 7.64 and cutting the result at two percentiles. Half the variance in the underlying score is unrecoverable by any function of the features, because it is not a function of the features. The Bayes-optimal classifier — one that knows the generating equation exactly — reaches 61.04%. The Random Forest reaches 60.39%, which is 98.9% of that limit and 95.9% of the accuracy available above the majority-class baseline.

The model is very nearly the best model that can exist for this problem. What remains is not error to be engineered away; it is the noise term, and no algorithm can see through it.

Two pieces of evidence make this more than an arithmetic coincidence. The model's predicted class distribution (31.4 / 56.2 / 12.5) tracks the Bayes rule's (32.9 / 54.4 / 12.7) rather than the true distribution (35 / 45 / 20), which means it has reproduced the Bayes rule's characteristic retreat toward the central class near a noisy boundary. And 1,706 of its 1,743 errors fall between adjacent bands, with only 37 records confusing *Low* with *High*. A model that had learned nothing would scatter its mistakes. This one places them exactly where the noise places the class boundary.

This kind of analysis is available only because the data is synthetic. It is the one genuine methodological advantage the synthetic approach confers, and it is worth being explicit that it is purchased at a steep price, discussed in Section 24.5.

24.2 The Model Is Calibrated Against the Wrong Objective

The Random Forest identifies high-risk individuals with precision 0.6898 and recall 0.4295. It is usually right when it says *High*; it says *High* for fewer than half of those who are.

For a risk-screening instrument this is the wrong way round. The cost of telling a genuinely high-risk person that they are medium-risk is that they take on exposure they cannot absorb. The cost of the reverse error is an unnecessarily conservative recommendation. These costs are plainly asymmetric, and the model's operating point optimises for neither — it optimises for accuracy, which weights them equally.

Section 23.3 supplies the remedy. Macro ROC AUC (0.7712) exceeds weighted ROC AUC (0.7461), an inversion which tells us that the model *ranks* high-risk individuals well even though the *argmax* rule rarely selects that class for them. The information is present in the probability vector and is thrown away by taking the maximum. Thresholding on `P(High)` at a value below 0.5 — chosen to hit a target recall — would trade precision for recall along a curve the model already supports. The interface displays a confidence figure, so the machinery is in place; only the decision rule needs changing.

That this was not done is a limitation, not a discovery. But it demonstrates something about evaluation practice: a model can be at its Bayes limit on accuracy and still be badly configured for the task it serves.

24.3 Failures That Metrics Cannot See

The most instructive result of this project is that its worst defects produced no bad numbers.

Feature engineering was written twice — once for training, once for serving — and the two implementations disagreed on the scale of `savings_rate`, the denominator of `debt_to_income_ratio`, the direction of `age_income_ratio`, the spelling and cardinality of four ordinal maps, and the entire formula for both composite scores. The test set was evaluated through the training path. Every reported metric was therefore correct, and every live prediction was wrong. A user reporting a 10% savings rate was assessed *Medium* where the training scale placed them at *High*, and `P(High)` was not even monotone in the savings rate.

Separately, the questionnaire's third insurance category was silently converted to a null by `pandas.read_csv` and imputed to the mode, so a third of the dataset carried a category it had never held. The explainability module discarded the sign of every SHAP value, so the interface's "red = decreases risk" legend rendered a distinction the data no longer contained. The frontend submitted to the non-persisting endpoint, so the database the analytics dashboard aggregated was never written to by the application.

Five of these six defects share a structure: each lived in the *agreement between two representations of one thing* — two implementations of a feature, two spellings of a category, two endpoints with the same shape, a colour legend and the values it coloured. None was a mistake inside a function. Every function was individually correct.

Unit tests do not find defects of this kind, because a unit test examines one representation. The eleven tests in this repository all passed throughout. What found them was checking one representation against another: routing training rows through the serving preprocessor and comparing the output column by column, reading the CSV as raw text instead of through pandas, asking whether any SHAP value returned by the API was ever negative.

The engineering conclusion is that the appropriate response is not more unit tests but a different kind of test. The equivalence check of Section 9.7 — asserting that the serving path and the training path produce identical feature vectors for the same input — would have caught the five feature defects on the day they were written. It runs in under a minute. It is not yet in the suite, and adding it is the single highest-value change available to this codebase.

The structural response is stronger still, and it was adopted: `preprocess.py` now emits every constant it fits, and `inference.py` reads them. The two paths can no longer disagree about a value because only one of them holds a value. Duplication was removed rather than tested.

24.4 A Negative Result Worth Reporting

The investment-preference model achieves 0.2586 accuracy against a 0.2869 majority baseline and a ROC AUC of 0.4949. It is worse than a constant and ranks at chance.

The temptation, on obtaining such a result, is to tune harder, try another algorithm, or quietly drop the component. None would have helped, and the reason is instructive. Reading the generator shows that `investment_preference` is sampled from one of three fixed probability vectors selected by `risk_category` alone. No feature enters the draw. The Bayes ceiling is 0.3675 given perfect knowledge of the risk band, and roughly 0.3210 given the 60.4%-accurate band the system can actually infer.

The model was never going to work, and the diagnosis required reading forty lines of data-generation code rather than running another experiment. This is worth stating as a methodological point: when a model fails, the generating process is a more informative place to look than the hyperparameter grid.

Two cautions attach. This says nothing about whether investment preference is predictable from behavioural attributes in reality — the finding concerns a synthetic label, and a real one would carry real dependence on income, age and experience. And the system still exposes the field through its API and renders it in the interface, which is a defensible choice only because this report states plainly that the value is meaningless.

24.5 The Cost of Synthetic Data

Every result above rests on data that no person supplied.

The gains are real. The generating function is known, which is what permits a Bayes ceiling to be computed at all — an analysis unavailable for any real dataset. Twenty-two thousand records were obtained without ethics approval, recruitment or expense. Correlation structure was specified rather than hoped for.

The losses are larger, and three of them bear directly on the interpretation of the results.

The relationships the model discovers are the relationships the generator was written to contain. Section 12.3 reports a correlation of +0.711 between savings rate and financial decision score; this is not a finding about savers, it is a restatement of the formula. When Section 21.5 observes that every feature entering `risk_score` appears in the top eight SHAP ranks, it is confirming that a Random Forest can recover a weighted sum, which was not in doubt.

No lifestyle feature reaches the top ten, and no lifestyle feature appears in `risk_score`. The system was built on the premise, set out in Section 13.6, that discretionary spending reveals risk preferences that direct questions do not. The dataset cannot test that premise, because its author did not encode it. The entire behavioural-economics motivation of this project is, on this data, untestable.

And the reported accuracy measures recovery of a known function, not prediction of human behaviour. A real population's risk band would depend on attributes not collected, would carry measurement error in the self-reports, and would exhibit relationships no generator anticipated. The 60.4% would not transfer, in either direction.

What the project does demonstrate is that the pipeline — elicitation, validation, preprocessing, inference, attribution, persistence, presentation — functions end to end and that its components agree with one another. That is a systems result, and it is a real one. It is not an empirical result about financial behaviour, and Section 26 states so.

24.6 Explanation Was the Weakest Link

The stated purpose of the explainability module was to restore, for a learned model, the per-answer transparency that a rule-based rubric provides for free. It falls short on three counts, and the shortfalls compound.

The attribution reaching the user is a magnitude, not a direction, because the serving code takes an absolute value and averages across classes. A user cannot learn from it whether their credit score helped or hurt.

The interface tells them it can. The chart is captioned "*Blue = increases risk score, Red = decreases risk score*" over data in which no value is ever negative. A caption that describes a distinction the data does not carry is worse than no caption, and it is the one defect in this system that actively misinforms rather than merely underperforming.

And the ranking itself is distorted by redundancy. `savings_ratio` and `expense_ratio` correlate at exactly -1.000000 ; `savings_rate` is a noisy third copy; `financial_discipline_score` weights `savings_rate` at 0.40. One underlying quantity occupies four of the top five rows, and its apparent importance is divided among them. The reader of the chart underestimates how completely a single number determines this model's output.

None of the three affects accuracy. All three affect the only thing the module exists to deliver. A model that is right for reasons it cannot articulate is, for this application, not obviously preferable to the rubric it replaced — and Section 8.5 claimed otherwise. That claim is not sustained by the implementation as it stands, and the fixes required, set out in Sections 21.4 and 22.5, are small.

Figure 24.1 — *Precision–recall trade-off for the `High` class*, sweeping the decision threshold on `P(High)` from 0.1 to 0.9, with the argmax operating point marked. This substantiates the argument of Section 24.2 and is the figure most likely to be asked about. Place there.

Table 24.1 — *Defect taxonomy*. Six rows, one per defect from Section 24.3. Columns: defect, where it lived, whether any metric was affected, whether any existing test could have caught it, how it was found. The pattern in the fourth column — uniformly "no" — is the argument.

25. Advantages

The advantages below are claimed only where the implementation supports them. Where a stated design goal was not achieved, it appears in Section 26 instead of here.

25.1 Evaluation Against a Derived Ceiling

The system's evaluation does not compare accuracy against perfect classification. Because the label's generating function is documented, the Bayes-optimal accuracy can be computed exactly: 0.6104. The tuned Random Forest attains 0.6039, which is 98.9% of that limit and 95.9% of the accuracy available above the majority-class baseline of 0.4500.

This reframes a figure that looks weak in isolation. It also supplies a falsifiable claim: had the model scored materially above 0.6104, that would have been evidence of leakage rather than skill, and the report would have had to explain it.

Two independent checks confirm the model has converged on the Bayes decision rule rather than coincidentally matching its accuracy. Its predicted class distribution (31.4 / 56.2 / 12.5) tracks the Bayes rule's (32.9 / 54.4 / 12.7) rather than the true distribution (35 / 45 / 20), reproducing the characteristic retreat toward the central class near a noisy threshold. And 1,706 of its 1,743 errors fall between adjacent bands; only 37 records, 0.84% of the test set, confuse *Low* with *High*.

25.2 Training and Serving Provably Agree

Feature engineering is defined once. `preprocess.py` writes every constant it fits — ordinal maps, IQR clip bounds, min–max normalisation ranges, imputation medians — to `preprocessing_params.json`, and `inference.py` reads that artefact rather than restating the values. No encoding constant is hardcoded in the serving path.

The equivalence is verified, not assumed. Three hundred randomly drawn training records routed through the serving preprocessor produce feature vectors agreeing with the training pipeline's to within 10^{-16} across all thirteen derived and encoded columns. Held-out records routed through the full serving path produce predictions identical to feeding the model its training-format features directly, on every row.

The accuracy reported in Section 23 is therefore the accuracy a live user receives. Section 22.4 documents that this was not true before the repair, and that no metric in the report would have revealed it.

25.3 Interactions Without Loss of Per-Answer Attribution

A rubric is a sum of independent terms and cannot represent that a thin emergency fund matters more for a sole earner with dependants. A tree ensemble represents such conditionals natively [9], [13].

Historically this capability cost the per-answer transparency that made rubrics defensible. Exact Shapley values for tree models are computable in polynomial time [7], so attribution can be recovered inside a single HTTP request over a 200-tree ensemble, without sampling. The mechanism that restores the rubric's

chief virtue is present and correct at the point of computation — though Section 21.4 records that the serving code then discards part of what it computed.

25.4 Uncertainty Is Reported, Not Hidden

The classifier returns a probability across all three bands rather than a bare label, and the interface displays the probability of the predicted class. Where the model is unsure, the user is told.

This is more than presentational. Macro ROC AUC (0.7712) exceeds weighted ROC AUC (0.7461), which means the model *ranks* high-risk individuals well even though the argmax rule selects that class for fewer than half of them. The information needed for a more conservative operating point already exists in the payload the API returns; Section 24.2 explains how to use it.

25.5 Assessments Are Reproducible and Auditable

Every persisted assessment stores the twenty-six submitted inputs, the predicted band, the full probability vector, the three derived scores and the SHAP attributions. Any past prediction can be reconstructed and examined. Because inference is deterministic given its inputs — the noise term exists only in the data generator, not in the model — re-running a stored record reproduces its original result exactly.

The database records what the user asserted rather than what the system derived. Section 17.4 defends the resulting third-normal-form violation on precisely this ground: `savings_rate` need not equal the value implied by the income and expenses the user also reported, and an audit trail should preserve the discrepancy rather than normalise it away.

25.6 Modularity

The five stages — generation, preprocessing, training, serving, presentation — communicate through files and HTTP rather than through shared memory, so each can be inspected and re-run independently. Re-running `preprocess.py` after the parameter-persistence change left `processed_dataset.csv` byte-identical, which is why the trained model remained valid and no retraining was needed to repair the serving defects.

The frontend holds no thresholds, no scoring weights and no model. A model change requires no frontend release. The API contract is a checked-in artefact against which both the backend schemas and the frontend types are validated, and Section 18.1 records the six interface divergences that existed before that discipline was adopted.

25.7 Deployment

Three containers compose: PostgreSQL with a health check, the FastAPI service gated on that health check, and an nginx-served static bundle. SQLite is supported for development through the same ORM, with the one non-portable query — date truncation in the analytics aggregate — resolved by inspecting the active dialect rather than assuming a backend.

25.8 The Report Is Falsifiable

Six defects are documented in the sections that describe the components containing them, rather than collected in a footnote or omitted. Two objectives are marked not met. A secondary model is reported as failing, with a derivation showing that no model could have succeeded. Every numeric claim in this report was recomputed from the artefacts in the repository during writing, and three figures the author had previously asserted — the Bayes ceiling, the frontend bundle size, and the count of recommendation branches — were found wrong and corrected.

This is stated as an advantage because the alternative was available and was not taken. A report claiming a working explainability module, a functioning investment recommender and a clean end-to-end system would have been shorter, and would have been false.

26. Limitations

26.1 The Data Is Synthetic

This is the limitation from which most others follow, and it cannot be mitigated within the present design.

No person supplied a record. The 22,000 rows were produced by a generator whose conditional dependencies were written by the author. Every correlation the analysis recovers is a correlation the generator was written to contain. When Section 12.3 reports $r = +0.711$ between savings rate and financial decision score, that is not a finding about savers; it is a restatement of a formula. When Section 21.5 observes that every feature entering `risk_score` appears among the top eight SHAP ranks, it confirms that a Random Forest can recover a weighted sum — which was not in doubt.

Three consequences deserve separate statement.

The reported accuracy does not transfer. A real population's risk band would depend on attributes not collected, would carry measurement error in the self-reports, and would exhibit relationships no generator anticipated. The 60.4% figure describes recovery of a known function, in neither direction a prediction about people.

The project's own premise is untestable on its own data. The system was built on the reasoning, set out in Section 13.6, that discretionary spending reveals risk dispositions that direct questioning does not. But `risk_score` is a function of five financial quantities and no lifestyle attribute enters it. That no lifestyle feature reaches the top ten SHAP ranks is therefore a property of the generator, not a result about behaviour. The behavioural-economics motivation of this project is, on this data, unfalsifiable.

The Bayes-ceiling analysis is purchased with the same coin. Deriving an attainable ceiling requires knowing the generating function, which is possible only because the data was constructed. The analysis that Section 25.1 claims as the report's strongest contribution is available precisely because the data is weakest.

26.2 The Explainability Module Does Not Deliver What It Claims

This is the most serious defect in the system, because explanation is the feature that distinguishes it from the rubric it proposes to replace.

`TreeExplainer` returns signed, class-specific Shapley values. The serving code reduces them with `np.abs(sv[0]).mean(axis=1)`, taking the absolute value and averaging across the three classes. What reaches the user is a magnitude — how much a feature mattered, in any direction, for any class — not an attribution to their predicted band. Across 120 attributions sampled from twelve varied inputs, none was negative.

The interface nonetheless captions its chart "*Blue = increases risk score, Red = decreases risk score.*" No bar can ever be red. For a record with a credit score of 720, that feature is rendered as increasing risk. This is the one defect in the system that actively misinforms rather than merely underperforming, and Rudin's objection to post-hoc explanation — that it may look authoritative while being unfaithful — describes it exactly [15].

The repair is two lines, given in Section 21.4. It was not applied because it would invalidate every SHAP figure in Sections 14, 21 and 22.

26.3 Redundant Features Corrupt the Attribution

`savings_ratio` and `expense_ratio` are exact complements: correlation -1.000000 , with $|\text{savings_ratio} + \text{expense_ratio} - 1|$ never exceeding 4.4×10^{-16} . `savings_rate` is a third, noisier encoding of the same quantity, correlating at 0.944. `financial_discipline_score` weights `savings_rate` at 0.40.

One underlying quantity therefore occupies four of the top five rows of the SHAP ranking, and ranks four and five carry identical importance to four decimal places. Accuracy is unaffected — a Random Forest is untroubled by collinearity — but a reader of the chart would substantially underestimate how completely a single number determines the model's output.

26.4 A Category Was Destroyed During Loading

`insurance_coverage` is generated with three levels. `pandas.read_csv` treats the bare string `None` as a null, so 7,264 records — 33.0% of the dataset — arrived as missing and were imputed to the mode, `Basic`.

The effect on accuracy is negligible, and measured: retraining with the category preserved gives weighted F1 0.5915 against 0.5955, a difference within the ± 0.0032 cross-validation spread. Insurance never enters `risk_score`, so the classifier lost nothing it was using.

The effect elsewhere is real. `behavioral_composite_score` weights insurance at 0.20 and is therefore overstated for a third of records. The fitted ordinal map has two levels, so a live user selecting "no insurance" is silently encoded as the training median. The system accepts an input it cannot represent.

26.5 The Interface Accepts Inputs the Model Cannot Represent

Every numeric field except `credit_score` admits a wider range than the training data spans. The form takes `investment_experience_years` up to 50 against a training support of 0–15, and `age` up to 100 against 18–75. Values are clipped to the fitted IQR bounds before any feature is derived, which makes the prediction well-defined but not valid: an out-of-range input receives the prediction the model would have made at the boundary.

`savings_rate` is declared non-negative, though the training data runs to -10% . A user whose expenses exceed their income cannot express that state and must enter zero.

`credit_score` is clipped at a lower bound of 660.5 by the IQR rule, discarding the variation among poor-credit individuals — exactly where the feature is most discriminative. This is why `credit_score` ranks eighth in SHAP despite carrying weight $+25$ in the generating function.

26.6 The Model Is Calibrated Against the Wrong Objective

High risk is predicted with precision 0.6898 and recall 0.4295. For a screening instrument the asymmetry runs the wrong way: failing to warn a genuinely high-risk person is costlier than an over-conservative recommendation, and the argmax rule weights the two errors equally.

The remedy exists and was not applied. Macro AUC exceeds weighted AUC, meaning the ranking information is present; thresholding on `P(High)` below 0.5 would trade precision for recall along a curve the model already supports.

26.7 There Is No Security Posture

No authentication, no authorisation, no sessions, no tokens. `SECRET_KEY` is declared in the settings model and consumed by no code path. The `users` table is created, related to `assessments`, and never written to.

`GET /assessments` returns every stored assessment, including all twenty-six financial fields of each, to any caller. The audit middleware persists the complete request body — income, expenses, debt, credit score — to a table with no access control. CORS restricts two localhost origins, which constrains browsers and not clients.

This was a scoping decision, and on synthetic inputs it is harmless. The system is not deployable against real financial disclosures without an authentication layer, an authorisation model for the history and analytics endpoints, and a retention policy for the audit log.

26.8 Defects in the Deployed Backend

`GET /model/info` reads an absolute path from the development machine and therefore returns `{"error": ...}` with HTTP 200 under Docker Compose, the documented deployment path.

The fallback in `ml_service` that locates the inference module when `/ml` is absent ascends one directory too many and resolves to a path that does not exist. The defect is masked under Docker, where the bind mount always exists, and surfaces when the service is run directly, where every prediction returns 503.

The audit middleware describes its database write as "non-blocking best-effort". It is best-effort; it is not non-blocking. A synchronous SQLAlchemy commit executes inside an `async` coroutine and occupies the event loop on every request.

26.9 Testing Is Thin

Eleven tests exist: five unit tests over `predict()` and six integration tests against a running service. There are no tests for the preprocessing scripts and none for the frontend.

More importantly, none of the eleven could have caught any of the six defects in this report, because five of the six lived in the *agreement between two representations of one thing* rather than inside any function. The equivalence check of Section 9.7 — asserting that the serving path and the training path compute identical feature vectors — would have caught five of them on the day they were written. It runs in under a minute and is not in the suite.

The integration tests exercise SQLite while the deployment uses PostgreSQL. The dialect-specific failure in `/analytics`, described in Section 18.7, passed its tests and returned HTTP 500 in production.

26.10 Methodological Gaps

No logistic-regression baseline was fitted, so the report cannot say whether the non-linear models earned their complexity. Given that the generating function is a weighted sum, a linear model might have performed comparably, and its absence is a real weakness in Section 15.

The published model-comparison table reports a tuned Random Forest against four untuned competitors, because `train.py` overwrites the winner's metrics after tuning. Selection was made on untuned scores and is unaffected, but the table overstates the margin.

The neural network was evaluated on unscaled inputs, which disadvantages it. As a control that lost by a wide margin this does not affect the selection, but the reported figure understates what a properly scaled network would achieve.

No probability calibration was assessed. The interface displays a confidence value; whether a stated confidence of 0.7 corresponds to being correct 70% of the time is unmeasured.

No latency benchmark was performed. Each request constructs a fresh `TreeExplainer` rather than reusing one, and performs a synchronous audit write.

26.11 The Recommendations Are Not Grounded

Nine bias rules and eleven recommendation branches are implemented as deterministic functions of the raw inputs and the predicted band. They are heuristics drawn from the behavioural-finance vocabulary, not inferences. None was validated against labelled data, and no claim is made that a user flagged with *present bias* exhibits it. This is the component of the system with the weakest evidentiary basis, and it is the one whose output most resembles advice.

27. Future Scope

The items below are ordered by the ratio of value delivered to effort required, not by ambition. The first four are corrections to defects this report documents, and none takes more than an afternoon.

27.1 Immediate Corrections

Restore the sign of the SHAP attributions. Two lines in `_get_shap_values`. Instead of averaging absolute values across classes, select the column of the predicted class and preserve its sign:

```
k = int(model.predict(X_df)[0])
sv_row = sv[0][:, k]
```

The interface is already written to render negative values in red. This is the first thing that should be done, because it is the difference between an explainability module and a bar chart of magnitudes, and because the current caption misinforms. Every SHAP figure in Sections 14, 21 and 22 must then be regenerated.

Promote the equivalence check to a regression test. The comparison described in Section 9.7 — routing training records through the serving preprocessor and asserting the feature vectors match — runs in under a minute and would have caught five of this project's six defects on the day they were introduced. It belongs in `tests/`, running in continuous integration, and it is the single highest-value change available to this codebase.

Remove `expense_ratio`. It is the exact complement of `savings_ratio` (correlation -1.000000) and contributes nothing but a split of the SHAP attribution across two identical rows. Removing it leaves accuracy unchanged and makes the explanation honest.

Fix the two backend path defects. `GET /model/info` should resolve its metadata path relative to `ML_PATH` rather than to an absolute location on the author's machine. The local-development fallback in `ml_service` should ascend three directory levels rather than four.

27.2 Making the Model Fit Its Purpose

Threshold on `P(High)` instead of taking the `argmax`. Macro AUC (0.7712) exceeds weighted AUC (0.7461), which establishes that the model ranks high-risk individuals well even though the `argmax` rule selects that class for fewer than half of them. Recall on *High* is 0.4295 against precision 0.6898. Choosing a threshold to hit a target recall — say 0.75 — would trade precision the application can afford for recall it cannot. The probability vector is already in the API response; only the decision rule needs changing.

Assess calibration. The interface displays a confidence figure. Whether a stated 0.7 corresponds to being right 70% of the time is unmeasured. A reliability diagram over the test partition, and Platt scaling or isotonic regression if the model proves miscalibrated, would make the displayed number mean what it appears to mean.

Fit a logistic-regression baseline. The generating function is a weighted sum, so a linear model may perform comparably to the ensemble. Without that comparison the report cannot claim the non-linear models earned their complexity, and Section 26.10 concedes as much. The baseline may well win, and that

would be a finding.

27.3 Repairing the Data

Preserve the `None` insurance category. Read the CSV with `keep_default_na=False` and restore the three-level ordinal map. Accuracy shifts by less than one cross-validation standard deviation, but `behavioral_composite_score` becomes correct for the third of records currently misattributed, and the live API stops mapping a user who reports no insurance onto the training median.

Rewrite the `investment_preference` generator. As Section 22.3 establishes, the label is currently sampled from a distribution conditioned on `risk_category` alone, giving it a Bayes ceiling of 0.3675 and making the secondary model unsalvageable. Making the draw depend on age, investment experience, luxury spending and risk band would give the target learnable structure. Every downstream number changes, so this belongs in a new iteration rather than a patch.

Reconsider the IQR clip on `credit_score`. The rule truncates at 660.5, discarding variation among poor-credit individuals precisely where the feature discriminates. This is why a variable carrying weight +25 in the generating function ranks eighth in SHAP. A wider bound, or clipping only the upper tail, would recover it.

Widen the label's signal-to-noise ratio, or say why not. The noise term at $\sigma = 8.0$ against a signal at $\sigma = 7.64$ caps achievable accuracy at 0.6104. This was almost certainly not deliberate. A generator with less noise would produce a more discriminative model, at the cost of a less interesting Bayes-ceiling analysis.

27.4 Toward Real Data

The largest limitation, and the hardest to address. Sections 5.2 and 26.1 argue that no ground truth for risk-band suitability exists even in principle, because no institution records years afterward whether an assigned band was correct.

Two tractable directions exist. A **small validation study** — 100 to 200 respondents completing both the form and the validated Grable–Lyttton instrument [3] — would not provide a training label, but would show whether the system's bands correlate with an instrument the literature accepts. That is a modest, checkable claim, and it requires ethics approval and time rather than novel methodology.

A **proxy-label study** would substitute an observable outcome for the unobservable one: portfolio volatility actually held, or realised drawdown tolerance during a market decline. This changes the research question from *what band suits this person* to *what behaviour does this person exhibit*, which is answerable. It is the direction in which this work would have to move to make any empirical claim at all.

Until one of these is done, the behavioural-economics premise stated in Section 13.6 — that lifestyle spending reveals dispositions self-report conceals — remains untested, because the synthetic label was constructed without reference to any lifestyle attribute.

27.5 Engineering the System

Authentication and authorisation. No principal exists, the `users` table is never written, and `GET /assessments` returns every stored financial disclosure to any caller. Session-based authentication would populate the vestigial table and permit the history and analytics endpoints to be scoped to their owner.

Audit-log policy. The middleware persists complete request bodies to an unprotected table and does so with a synchronous commit inside an asynchronous handler. The write should move to a background task, and the body should be redacted or omitted.

Schema migrations. `backend/alembic/` is empty and the schema is materialised by `metadata.create_all()`, which silently ignores drifted column definitions. Generating an initial revision is a prerequisite to any change against a database holding data.

Indexes. No secondary index exists. The history listing orders `assessments` by `created_at`, and the analytics aggregations group `predictions` by `risk_category` and `investment_preference`. Those three columns are where load will first show.

Reuse the `TreeExplainer`. It is constructed per request against a 200-tree ensemble. Building it once at startup would remove the dominant cost of a prediction. No latency benchmark exists to quantify the gain, and establishing one is a precondition for claiming it.

Code-split the frontend. The bundle is 922.99 kB, 279.68 kB gzipped, dominated by Recharts and its D3 dependencies. Deferring the analytics and results routes behind `React.lazy` would bring the initial chunk under Vite's 500 kB threshold.

27.6 Beyond the Present Design

Interaction effects are computable for tree models at the same polynomial cost as the attributions themselves [7], and the interface presents none. Showing that a low emergency fund matters more for a user with dependants would demonstrate the capability that justifies replacing an additive rubric — the capability Section 25.3 claims and Section 5.2 concedes the synthetic label barely exercises.

Counterfactual explanation — *if your savings rate were 22% rather than 12%, your band would change* — is a more useful artefact for an individual than an attribution, and is what a user actually wants from a risk assessment.

And the incumbent deserves a fair test. Rudin's position is that a high-stakes decision should use an inherently interpretable model rather than an explained black box [15]. A scored rubric fitted to this data, rather than asserted, would be interpretable by construction and might lose very little accuracy against a label that is a weighted sum. Section 4.5 concedes that on transparency the rubric currently leads. Benchmarking against it would settle whether anything was gained.

28. Conclusion

28.1 What Was Built

An end-to-end system that elicits twenty-six demographic, financial and lifestyle attributes through a four-step web form, classifies the respondent into one of three financial-risk bands with a tuned Random Forest, attributes the outcome with Shapley values, persists the assessment to PostgreSQL, and renders the result through a React dashboard. Three services compose under Docker. Seven REST endpoints are validated by schemas that also generate their own documentation.

The dataset is synthetic: 22,000 records produced by a generator with documented conditional dependencies, because no public dataset joins these attributes to a behavioural risk label, and Section 5.2 argues no such label exists even in principle.

28.2 What Was Found

The model operates at the Bayes limit. Because the label's generating function is known, the maximum accuracy attainable by any classifier can be computed rather than guessed. It is 0.6104, the noise term having a standard deviation of 8.0 against a signal of 7.64. The Random Forest attains 0.6039 — 98.9% of the ceiling, and 95.9% of the accuracy available above the majority baseline. Its predicted class distribution tracks the Bayes rule's rather than the truth, and 1,706 of its 1,743 errors fall between adjacent bands. Read against 1.0 the result looks mediocre; read against what the data permits, the model recovers nearly all recoverable signal.

A negative result is reported in full. The secondary model predicting investment preference scores 0.2586 against a majority baseline of 0.2869 and ranks at chance. Reading the generator shows the label is drawn from a distribution conditioned on the risk band alone, giving a ceiling of 0.3675 under perfect knowledge. No algorithm, no tuning and no additional feature could have rescued it. The diagnosis came from forty lines of data-generation code, not from another experiment.

The worst defects produced no bad numbers. Feature engineering was implemented twice — once for training, once for serving — and the two disagreed on the scale of `savings_rate`, the denominator of `debt_to_income_ratio`, the direction of `age_income_ratio`, the spelling of four ordinal maps, and the formula for both composite scores. Every reported metric stayed correct, because the test set was evaluated through the training path. Every live prediction was wrong. Separately, a third of the `insurance_coverage` column was destroyed by `pandas` on load; the explanation pipeline discarded the sign of every attribution before display; and the form submitted to the non-persisting endpoint, so nothing a user entered ever reached the database.

Five of those six defects share one structure. None was a mistake inside a function; each lived in the *agreement between two representations of the same thing* — two implementations of a feature, two spellings of a category, two endpoints of the same shape, a colour legend and the values it coloured. All eleven tests passed throughout. What exposed them was checking one representation against another. This is the hazard Sculley and colleagues describe as the hidden technical debt of machine learning systems [8], and it is the finding of this project most likely to generalise.

28.3 What Was Not Achieved

Seven of ten objectives were met, two of them only after defects found while writing this report were repaired.

Two were not met. The explainability module returns absolute magnitudes averaged across classes rather than signed attributions to the predicted band, so the interface's caption — "*Blue = increases risk score, Red = decreases risk score*" — describes a distinction its data does not carry, and no bar is ever red. This is uncomfortable, because explanation is the sole feature distinguishing this system from the rubric it proposes to replace. The repair is two lines and is deferred only because it invalidates every SHAP figure already generated. And the investment-preference objective was unattainable by construction.

The redundancy among `savings_rate`, `savings_ratio` and `expense_ratio` — the latter two correlating at exactly -1.000000 — splits one dominant signal across four of the five top-ranked features, so the explanation understates how completely one quantity determines the model. Accuracy is untouched; interpretation is not.

28.4 What It Does Not Show

Nothing in this report is evidence about human financial behaviour. Every correlation recovered is one the generator was written to contain. The project's own premise — that lifestyle spending reveals dispositions self-report conceals — is untestable here, because no lifestyle attribute enters the generating function. That no lifestyle feature reaches the top ten SHAP ranks says something about the generator and nothing about people.

The Bayes-ceiling analysis, the report's strongest methodological contribution, is available only because the data is synthetic. The analysis and the limitation are the same fact seen from two sides.

28.5 Closing Assessment

Judged as science, the project establishes nothing about behavioural finance and applies no novel method.

Judged as engineering, it delivers a working, containerised, verified pipeline and an evaluation more careful than the result required — accuracy reported against a derived ceiling rather than against perfection, serving-time feature computation proved identical to training-time, and a failing component reported as failing rather than quietly removed.

The most durable lesson is the least glamorous. A test set drawn from the training pipeline certifies the estimator and says nothing about the code that will serve it. The accuracy in a report and the accuracy a user receives are different quantities, and in this system they differed for most of its existence, silently, while every metric agreed. Establishing that they now coincide took one comparison, sixty seconds of compute, and no new theory — and it should have been the first thing written, not among the last.

29. References

IEEE style, numbered in order of first citation. Every entry was checked against a primary or authoritative secondary source during preparation of this report; Section 29.1 records what was confirmed and what was not. No reference is included that could not be located.

- [1] D. Kahneman and A. Tversky, "Prospect theory: An analysis of decision under risk," *Econometrica*, vol. 47, no. 2, pp. 263–291, Mar. 1979.
- [2] N. Barberis and R. Thaler, "A survey of behavioral finance," in *Handbook of the Economics of Finance*, vol. 1, G. M. Constantinides, M. Harris, and R. M. Stulz, Eds. Amsterdam, The Netherlands: Elsevier, 2003, ch. 18, pp. 1053–1128.
- [3] J. Grable and R. H. Lytton, "Financial risk tolerance revisited: The development of a risk assessment instrument," *Financial Services Review*, vol. 8, no. 3, pp. 163–181, 1999.
- [4] J. Klement, Ed., *Risk Profiling and Tolerance: Insights for the Private Wealth Manager*. Charlottesville, VA, USA: CFA Institute Research Foundation, 2018.
- [5] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, "Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research," *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, Nov. 2015.
- [6] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Long Beach, CA, USA, 2017, pp. 4765–4774.
- [7] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, Jan. 2020, doi: 10.1038/s42256-019-0138-9.
- [8] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, Montréal, QC, Canada, 2015.
- [9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, 2016.
- [11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Long Beach, CA, USA, 2017.
- [12] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, Montréal, QC, Canada, 2018, pp. 6639–6649.
- [13] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on tabular data?," arXiv:2207.08815 [cs.LG], Jul. 2022.

[14] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?": Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, 2016.

[15] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, May 2019, doi: 10.1038/s42256-019-0048-x.

29.1 Verification Notes

Working material for the author. Delete before submission.

Verified against a primary source — publisher page, arXiv abstract, or official proceedings listing — for title, authorship and venue: [6], [7], [8], [9], [10], [11], [12], [13], [14].

Verified against a primary source including full pagination: [3] via the RePEc listing for *Financial Services Review* 8(3); [5] via the RePEc listing for *European Journal of Operational Research* 247(1).

Verified against concurring secondary sources: page ranges for [1], [2], [9], [7], [15]; venue, editor and year for [4].

Pagination deliberately omitted, because it could not be confirmed from a primary source: [8], [10], [11], [14]. IEEE style permits a conference reference without pagination. The conventional values are widely cited — KDD '16 pp. 785–794 for [10], NIPS 2017 pp. 3146–3154 for [11], KDD '16 pp. 1135–1144 for [14] — but they are recorded here only as a note and must be confirmed against the ACM Digital Library or the NeurIPS proceedings before insertion.

One discrepancy unresolved. Sources disagree on the final page of [1], giving either 263–291 or 263–292. The Econometric Society's own listing returned HTTP 403 during preparation. The value 263–291 is used above, being the more commonly cited, and should be confirmed against the journal itself.

Claims checked, not assumed. The abstract of [8] enumerates boundary erosion, entanglement, hidden feedback loops, undeclared consumers, data dependencies and configuration debt. It does **not** use the phrase *training–serving skew*. Section 3.5 and Section 5.2 therefore cite [8] for hidden technical debt and for the observation that most of a deployed ML system is not the model, and do not attribute the training–serving terminology to it.

Deliberate omissions. Two sources consulted on robo-advisory risk profiling were not cited, because their full text could not be retrieved and their claims could not be checked: an MDPI *Sustainability* article on content analysis of robo-advisor questionnaires, and a *Financial Services Review* retrospective on the Grable–Lytton scale. The argument they would have supported — that deployed questionnaires conflate risk capacity with risk tolerance — is instead attributed to [4], whose scope, editor and year were confirmed.

No reference in this list was reconstructed from memory. Where a detail could not be established, it was omitted rather than guessed.

30. IEEE-Format Research Paper

Formatting note. The text below is written to the IEEE conference template (two-column, 10 pt Times New Roman, `\documentclass[conference]{IEEEtran}`). Section numbering restarts here and is independent of the main report. Figure and table callouts are marked; sources for each are given in the note at the end.

Evaluating a Behavioural Risk-Assessment System Against Its Bayes Ceiling: Explainability, Serving Skew, and a Negative Result

Chirag Mali *Department of Computer Engineering* [Institution] [City, India] [email]

Abstract

Retail financial risk profiling remains dominated by questionnaires scored against fixed rubrics whose weights encode assumption rather than evidence and which, being additive, cannot represent interactions among a respondent's attributes. We replace the rubric with a tree-ensemble classifier and attempt to preserve per-answer transparency using Shapley attribution, then audit the resulting system end to end. On 22,000 synthetic records with twenty-six demographic, financial and lifestyle attributes, a tuned Random Forest attains accuracy 0.6039 and weighted F1 0.6006. Because the label is a threshold on a linear score perturbed by Gaussian noise of known scale, we derive the Bayes-optimal accuracy exactly: 0.6104. The model therefore reaches 98.9% of the attainable ceiling and 95.9% of the accuracy available above the majority-class baseline, and its predicted class distribution tracks the Bayes rule rather than the empirical one. We report a secondary model as a negative result: its target was generated conditioned on the risk band alone, bounding any classifier at 0.3675, and it scores 0.2586 against a 0.2869 baseline. Finally we characterise a class of defect invisible to model evaluation. Feature engineering implemented separately for training and for serving diverged in five places; every reported metric stayed correct while every live prediction was wrong, and all unit tests passed. We argue that verifying training-serving feature equivalence, and auditing the explanation pipeline rather than only the explanation method, belong in the standard evaluation apparatus.

Index Terms — behavioural finance, explainable AI, SHAP, Bayes-optimal accuracy, training-serving skew, negative results, synthetic data.

I. Introduction

Classical finance models the investor as an expected-utility maximiser. Kahneman and Tversky showed the description fails systematically: outcomes are evaluated as gains and losses against a reference point, losses weigh roughly twice as heavily as equivalent gains, and small probabilities are distorted [1]. Barberis and Thaler survey the consequences for market and individual behaviour [2].

Institutions nonetheless place clients on a risk spectrum using questionnaires scored by fixed rubrics; Grable and Lytton's thirteen-item scale is the canonical validated instance [3]. Two weaknesses are structural. The weights are asserted rather than estimated, so where an analyst's beliefs are wrong the instrument is wrong identically for every respondent. And an additive score cannot express interaction — that a thin emergency fund is more dangerous for a sole earner with dependants. Klement's survey adds a third: deployed instruments routinely conflate risk *capacity*, a balance-sheet fact, with risk *tolerance*, a psychological disposition [4].

Supervised learning addresses the first two. It is long established in credit scoring, where Lessmann *et al.* find tree ensembles consistently strongest [5]. It has not migrated to risk profiling, and the obstacle is not technical: a credit model predicts default, which is observed, whereas a risk band predicts suitability, which no institution ever labels.

A learned model also forfeits the rubric's transparency. Lundberg and Lee unified additive attribution methods under the Shapley value [6]; an exact polynomial-time algorithm for tree ensembles [7] makes per-request attribution cheap. Rudin objects that a post-hoc explanation is not the computation it describes and may mislead while appearing authoritative [15].

This paper contributes three results, none of which required a new algorithm.

1. **Evaluation against a derived ceiling.** Where a label's generating process is known, the Bayes-optimal accuracy is computable. We derive it (0.6104) and report the model's 0.6039 relative to it, rather than to unity.
2. **A negative result, diagnosed at the source.** A secondary classifier fails below its own majority baseline. Reading the generator — not further tuning — shows why no classifier could succeed.
3. **A defect class invisible to model evaluation.** Training and serving computed features differently in five places while all metrics and all tests remained green.

II. Related Work

Behavioural foundations. Prospect theory establishes reference dependence and loss aversion [1]; the survey literature catalogues overconfidence, herding and present bias as population regularities [2]. These are established by experiment, not as attributes inferable from a spending profile.

Measurement. Grable and Lytton validate an instrument psychometrically [3]. Klement documents the capacity/tolerance conflation as endemic [4].

Estimators. Breiman's Random Forest bounds generalisation error via tree strength and inter-tree correlation [9]. Gradient boosting refines this: XGBoost with a regularised objective and sparsity-aware splits [10], LightGBM with one-side sampling and histogram binning [11], CatBoost with ordered boosting

against target leakage [12]. Grinsztajn *et al.* find tree ensembles still ahead of neural networks on tabular data, attributing the gap to robustness against uninformative features and a bias toward axis-aligned boundaries [13].

Explanation. LIME fits a local interpretable surrogate [14]. Lundberg and Lee show LIME and others are special cases of additive attribution, within which the Shapley value uniquely satisfies local accuracy, missingness and consistency [6]. `TreeExplainer` computes it exactly in polynomial time for trees [7]. Rudin argues against post-hoc explanation for high-stakes decisions [15].

Systems. Sculley *et al.* frame production ML maintenance as technical debt, observing that most of a deployed system is not the learning algorithm [8]. The evaluation literature measures models; the debt accrues elsewhere.

III. Data and Method

A. Dataset

No public dataset joins twenty-six demographic, financial and lifestyle attributes to a behavioural risk label, and Section VII argues no such label exists in principle. We therefore generate 22,000 records under a fixed seed with explicit conditional dependencies: income conditioned on occupation and education, expenses on income and dependants, credit score on debt burden and savings, and so forth.

The risk label is a threshold on a continuous score,

$$\text{risk_score} = -30 \cdot s + 25 \cdot (1 - c/900) + 20 \cdot \min(d, 5)/5 - 15 \cdot (e/20) + 10 \cdot [f < 3] + \varepsilon$$

with `s` savings rate, `c` credit score, `d` debt-to-income, `e` investment experience, `f` emergency-fund months, and $\varepsilon \sim N(0, 8)$. Cuts at the 35th and 80th percentiles give classes at 35 / 45 / 20.

The noise dominates. The deterministic component has standard deviation 7.642 against a noise standard deviation of 8.000 — a signal-to-noise variance ratio of 0.912.

B. Preprocessing

Missing values imputed; nine numeric columns clipped at $1.5 \times \text{IQR}$; five ordinal and six nominal columns encoded; seven features derived (three ratios, two composites, two others), yielding 48 model features from 61 columns. Every constant fitted here — ordinal maps, clip bounds, min-max ranges, medians — is persisted to a JSON artefact consumed at serving time, for reasons Section VI-C makes clear.

C. Model Selection

Stratified 80/20 split under a fixed seed. Five classifiers trained with library defaults on the identical partition and compared on weighted F1, chosen over accuracy because a constant predictor achieves 0.45. The Random Forest led at $F1 = 0.5949$ and alone received a randomised hyperparameter search (20 candidates, 3-fold CV), reaching 0.6006.

(Table I — model comparison.)

D. Explanation

Exact Shapley values via `TreeExplainer` [7]: globally by averaging absolute values across the test partition, locally per request for the submitted row.

IV. System

A React/TypeScript client elicits the twenty-six attributes through a four-step form and holds no business logic. A FastAPI service validates against Pydantic schemas whose categorical fields are constrained to the exact training vocabulary, calls the inference module in-process (avoiding a cross-language serialisation boundary), persists inputs, prediction and attributions to PostgreSQL, and returns a probability vector. Three containers compose. Seven endpoints are documented by the same schemas that enforce them.

(Fig. 1 — three-tier architecture and data flow.)

V. Results

A. Classification Performance

Accuracy 0.6039, weighted F1 0.6006, one-vs-rest weighted ROC AUC 0.7461 on 4,400 held-out records. Five-fold CV F1 = 0.5996 ± 0.0032 .

(Table II — per-class precision, recall, F1.)

Per class, *High* carries the highest precision (0.6898) and the lowest recall (0.4295).

B. The Bayes Ceiling

Every quantity entering `risk_score` is available as a feature, so the deterministic component is exactly recoverable; ϵ is not. For a record with signal s , class probabilities follow from the noise CDF, and the Bayes-optimal rule takes their argmax. Recovering the thresholds numerically ($t_1 = -11.074$, $t_2 = 2.598$) so that induced proportions match the observed 35/45/20 and averaging the success probability over all records gives:

Reference	Accuracy
Majority class	0.4500
Random Forest	0.6039
Bayes ceiling	0.6104

The model attains **98.9%** of the ceiling and **95.9%** of the accuracy available above the baseline: $(0.6039 - 0.4500)/(0.6104 - 0.4500) = 0.9595$.

Two checks indicate convergence on the Bayes *rule*, not merely on its accuracy. Near a noisy threshold the optimal rule retreats toward the central class, and the model reproduces this:

Class	True	Bayes	Model
Low	35.0%	32.9%	31.4%
Medium	45.0%	54.4%	56.2%
High	20.0%	12.7%	12.5%

And errors respect ordinal structure: 1,706 of 1,743 errors are between adjacent bands; only 37 records (0.84%) confuse *Low* with *High*.

(Fig. 2 — accuracy against reference points. Fig. 3 — confusion matrix.)

C. A Negative Result

A secondary Random Forest predicting investment preference across five classes scores accuracy 0.2586, weighted F1 0.1892, ROC AUC 0.4949 — below the 0.2869 majority baseline and at chance in ranking.

The generator draws this label from one of three fixed probability vectors selected by `risk_category` alone; no feature enters. Hence

`ceiling | true band =  $\sum_k P(\text{band} = k) \cdot \max(p_k) = 0.3675$ ,`

and ≈ 0.3210 given the 60.4%-accurate band the system can infer. No algorithm, tuning budget or additional feature could have succeeded. The diagnosis required reading the data generator, not running experiments.

D. Training–Serving Skew

Feature engineering existed in two implementations. They disagreed on: the scale of `savings_rate` (fraction vs. percentage); the denominator of `debt_to_income_ratio` (annual vs. monthly income); the direction of `age_income_ratio`; the spelling and cardinality of four ordinal maps; and the formulae of both composite scores.

The held-out set was evaluated through the training path. **Every reported metric was correct. Every live prediction was wrong.** Predicted risk was not monotone in the savings rate: a user reporting 10% was assessed *Medium* where the training scale placed them at *High*.

The repair eliminated the duplication rather than fixing five instances: the training pipeline emits its fitted constants and the serving path reads them. Equivalence was then verified — 300 records agree across all thirteen derived features to 10^{-16} , and predictions agree with the training-format path on every held-out row.

E. Explanation Pipeline Failure

`TreeExplainer` returns signed, class-specific values. The serving code reduced them by `np.abs(sv[0]).mean(axis=1)` — absolute value, then averaged across classes. Across 120 attributions from twelve inputs, none was negative. The interface nonetheless captioned its chart "*Blue = increases risk, Red = decreases risk*"; no bar can ever be red.

Separately, `savings_ratio` and `expense_ratio` are exact complements ($r = -1.000000$), and `savings_rate` a noisy third copy ($r = 0.944$), so one quantity occupies four of the five top-ranked features and its importance is split among them.

The attribution *method* was sound. The *pipeline* was not, and nothing detected the difference — precisely the failure mode Rudin anticipates [15].

VI. Discussion

A. Sixty percent is not weak. Against unity it looks poor; against 0.6104 it is near-optimal. Reporting `A/A*` rather than `A` is available whenever a generating process is known, and is the discipline this paper presses.

B. Optimal accuracy, wrong objective. *High* recall of 0.4295 is the Bayes-optimal response to threshold noise — and the wrong operating point for a screening instrument, where under-warning is costlier than over-caution. Macro AUC (0.7712) exceeding weighted AUC (0.7461) shows the ranking information is present; thresholding `P(High)` below 0.5 would recover recall. A model can sit at its Bayes limit and still be misconfigured for its task.

C. Metrics do not see what breaks. Five of six defects lay not inside any function but in the *agreement between two representations of one computation* — two feature implementations, two spellings of a category, two endpoints of identical shape, a colour legend and the values it coloured. All eleven tests passed. Unit tests examine one representation. What found these was comparing representations: routing training rows through the serving preprocessor, reading the CSV as text rather than through `pandas`, asking whether any returned attribution was ever negative. This is the debt Sculley *et al.* describe [8], and we propose that a training–serving equivalence assertion belongs in the standard suite. Ours runs in under a minute and would have caught five of the six.

VII. Limitations

The data is synthetic, and this forfeits every empirical claim. Recovered correlations are those the generator was written to contain. The project's own premise — that lifestyle spending reveals dispositions self-report conceals — is untestable here, because no lifestyle attribute enters `risk_score`; that none reaches the top ten SHAP ranks is a fact about the generator. The Bayes-ceiling analysis, our strongest contribution, exists only because the data is synthetic: the contribution and the limitation are one fact viewed twice.

No ground truth for risk-band suitability exists even in principle, since no institution records whether an assigned band was later correct. The explanation defect of Section V-E is unrepaired, as correcting it invalidates the reported SHAP figures. No logistic baseline was fitted, so we cannot claim the non-linear model earned its complexity against a label that is a weighted sum. No calibration assessment and no latency benchmark were performed. The system has no authentication and is not deployable against real financial disclosures.

VIII. Conclusion

We built and audited an end-to-end behavioural risk-assessment system. The classifier reaches 98.9% of the Bayes-optimal accuracy its data permits, having converged on the Bayes decision rule rather than merely its accuracy. A secondary model is reported as a negative result, with a derivation showing its target carried no learnable structure. Most instructively, five defects that rendered every live prediction wrong left every reported metric correct and every test passing, because they lived in the agreement between two implementations rather than within either.

We conclude that a held-out test set drawn from the training pipeline certifies the estimator and says nothing about the code that will serve it; that an explanation pipeline requires auditing separately from the explanation method it invokes; and that accuracy is uninterpretable without the ceiling against which it is read.

References

Use the fifteen entries of Section 29, unchanged and in the same order. Citation numbers in this paper correspond exactly.

Preparation Note

Figures. Fig. 1 renders from the Mermaid source in Section 11. Fig. 2 is specified as Figure 23.3. Fig. 3 as Figure 22.1. A fourth figure — `P(High)` versus submitted `savings_rate`, before and after the skew repair (Figure 22.3) — is the most persuasive single artefact and should be included if space permits.

Tables. Table I from Section 15.4, annotating that the Random Forest row is the tuned model and the others are untuned. Table II from Section 23.2.

Length. As drafted this runs to roughly six pages in the IEEE conference template. Sections II and IV compress most readily; Sections V-B, V-C and V-D should not be cut, as they carry the contributions.

Before submission. Confirm the pagination gaps flagged in Section 29.1, resolve the page-range discrepancy in [1], and delete Section 29.1 itself.